Chương 3

Bảng trang (Page tables)

Bản dịch tiếng Việt của Chương 3 – tài liệu xv6 (RISC-V), có bổ sung giải thích và ví dụ minh họa. Phần bản dịch bám sát nguyên bản; các khung màu là phần giải thích thêm của người dịch (xanh dương: giải thích cơ chế/thuật toán; vàng: ví dụ số liệu; xanh lá: giải thích hình).

Bảng trang (page table) là cơ chế phổ biến nhất mà hệ điều hành dùng để cấp cho mỗi tiến trình một không gian địa chỉ và bộ nhớ riêng tư. Page table quyết định các địa chỉ bộ nhớ có ý nghĩa gì và phần nào của bộ nhớ vật lý được phép truy cập. Nó cho phép xv6 cô lập không gian địa chỉ của các tiến trình khác nhau và ghép kênh (multiplex) chúng lên một bộ nhớ vật lý duy nhất. Page table là một thiết kế phổ biến vì nó cung cấp một mức gián tiếp (level of indirection) cho phép hệ điều hành thực hiện nhiều "mánh khóe". Xv6 dùng một vài mánh như vậy: ánh xạ cùng một vùng nhớ (trang trampoline) vào nhiều không gian địa chỉ, và canh gác (guard) các stack của kernel và của user bằng một trang không được ánh xạ. Phần còn lại của chương này giải thích page table mà phần cứng RISC-V cung cấp và cách xv6 sử dụng chúng.

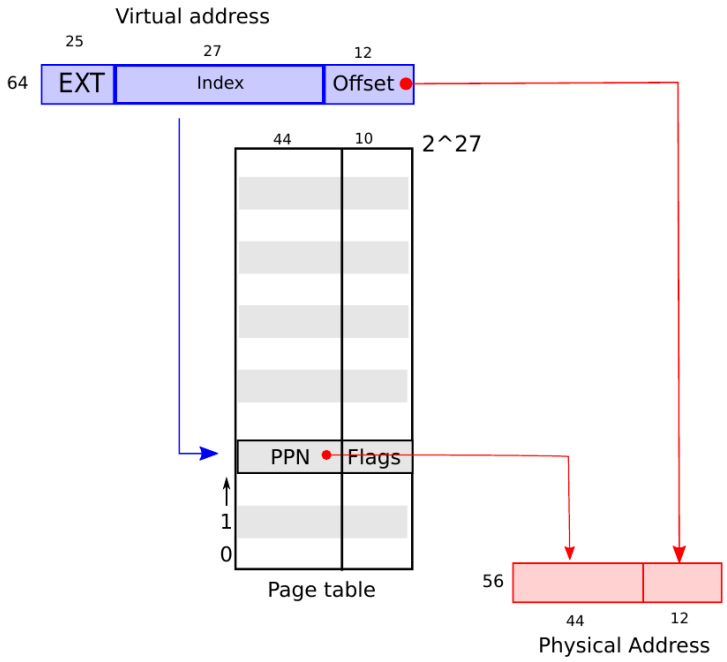
> **Giải thích thêm – Page table là gì, nhìn nhanh bằng một hình dung đời thường**
>
> Hãy tưởng tượng một tòa chung cư. Mỗi cư dân (tiến trình) có "số phòng" riêng theo cách đánh số của mình: phòng 101, 102, ... Hai cư dân khác nhau đều có thể có "phòng 101", nhưng đó là hai căn phòng thật khác nhau. Nhân viên quản lý tòa nhà (phần cứng MMU) có một **sổ tra cứu** (page table) để đổi "số phòng của cư dân" (địa chỉ ảo) thành "vị trí thật trong tòa nhà" (địa chỉ vật lý). Vì chỉ cần sửa sổ tra cứu là đổi được căn phòng thật, hệ điều hành có thể: (1) giấu phòng của người này với người kia (cô lập), (2) cho nhiều người cùng vào một phòng chung (chia sẻ, như trampoline), (3) để trống một số số phòng để bắt lỗi truy cập sai (guard page).

3.1 Phần cứng phân trang (Paging hardware)

Xin nhắc lại: các lệnh RISC-V (cả của user lẫn của kernel) thao tác trên **địa chỉ ảo** (virtual address). RAM của máy, hay còn gọi là bộ nhớ vật lý, được đánh chỉ số bằng **địa chỉ vật lý** (physical address). Phần cứng page table của RISC-V nối hai loại địa chỉ này với nhau bằng cách ánh xạ mỗi địa chỉ ảo thành một địa chỉ vật lý.

Xv6 chạy trên RISC-V theo chuẩn Sv39, nghĩa là chỉ 39 bit thấp của địa chỉ ảo 64 bit được dùng; 25 bit cao không dùng. Trong cấu hình Sv39 này, một page table của RISC-V về mặt logic là một mảng gồm 2^{27} (134.217.728) mục page table (page table entry – PTE). Mỗi PTE chứa một số trang vật lý (physical page number – PPN) 44 bit và một số cờ (flag). Phần cứng

phân trang dịch một địa chỉ ảo bằng cách dùng 27 bit cao trong 39 bit đó để đánh chỉ số vào page table nhằm tìm ra một PTE, rồi tạo ra một địa chỉ vật lý 56 bit mà 44 bit cao lấy từ PPN trong PTE còn 12 bit thấp được chép từ địa chỉ ảo ban đầu. Hình 3.1 cho thấy quá trình này với cái nhìn logic về page table như một mảng PTE đơn giản (xem Hình 3.2 để có bức tranh đầy đủ hơn). Page table cho phép hệ điều hành kiểm soát việc dịch địa chỉ ảo sang địa chỉ vật lý ở mức độ chi tiết là các khối được căn chỉnh (aligned) có kích thước 4096 (2^{12}) byte. Một khối như vậy được gọi là một **trang** (page).



Hình 3.1: Địa chỉ ảo và địa chỉ vật lý của RISC-V, với một page table logic được đơn giản hóa.

Giải thích hình – đọc Hình 3.1

Hình 3.1 là mô hình "phẳng" (một cấp) của việc dịch địa chỉ:

- **Thanh màu xanh phía trên** là địa chỉ ảo 64 bit, chia làm ba đoạn: **EXT** (25 bit cao nhất, không dùng để dịch – chỉ cần là bản sao của bit 38 theo quy định phần cứng), **Index** (27 bit) và **Offset** (12 bit thấp).
- **Index (27 bit)** được dùng như chỉ số mảng: cột chữ nhật ở giữa là page table gồm 2^{27} dòng, đánh số từ 0 (dưới cùng) lên trên. Mũi tên xanh chỉ tới đúng dòng được chọn.
- Mỗi dòng (PTE) rộng 54 bit có ý nghĩa: **44 bit PPN** và **10 bit Flags** (hình vẽ ghi 44 và 10). Mũi tên đỏ từ PPN dẫn sang ô **Physical Address** (56 bit) – nó trở thành 44 bit cao của địa chỉ vật lý.
- **Offset (12 bit)** không đi qua page table: mũi tên đỏ đi thẳng xuống và trở thành 12 bit thấp của địa chỉ vật lý. Vì thế byte thứ k trong một trang ảo luôn là byte thứ k trong trang vật lý tương ứng.

Tóm lại: **PA = (PPN của PTE) ghép với (Offset của VA)**, tức PA = PPN × 4096 + Offset. Mô hình này dễ hiểu nhưng tốn bộ nhớ, và Hình 3.2 sẽ cho thấy cách thực tế (ba cấp) khắc phục điều đó.

Ví dụ minh họa – kích thước thật của một page table "phẳng" và vì sao cần nhiều cấp

Một page table phẳng có 2^{27} PTE, mỗi PTE chiếm 8 byte (64 bit) nên cần $2^{27} \times 8 = 2^{30}$ byte = **1 GiB** cho **mỗi tiến trình**, kể cả khi tiến trình chỉ dùng vài trang. Nếu máy có 128 MB RAM như QEMU mặc định của xv6, ta không thể giữ một bảng như thế cho mỗi tiến trình. Đó là lý do phần cứng dùng cây ba cấp ở phần sau.

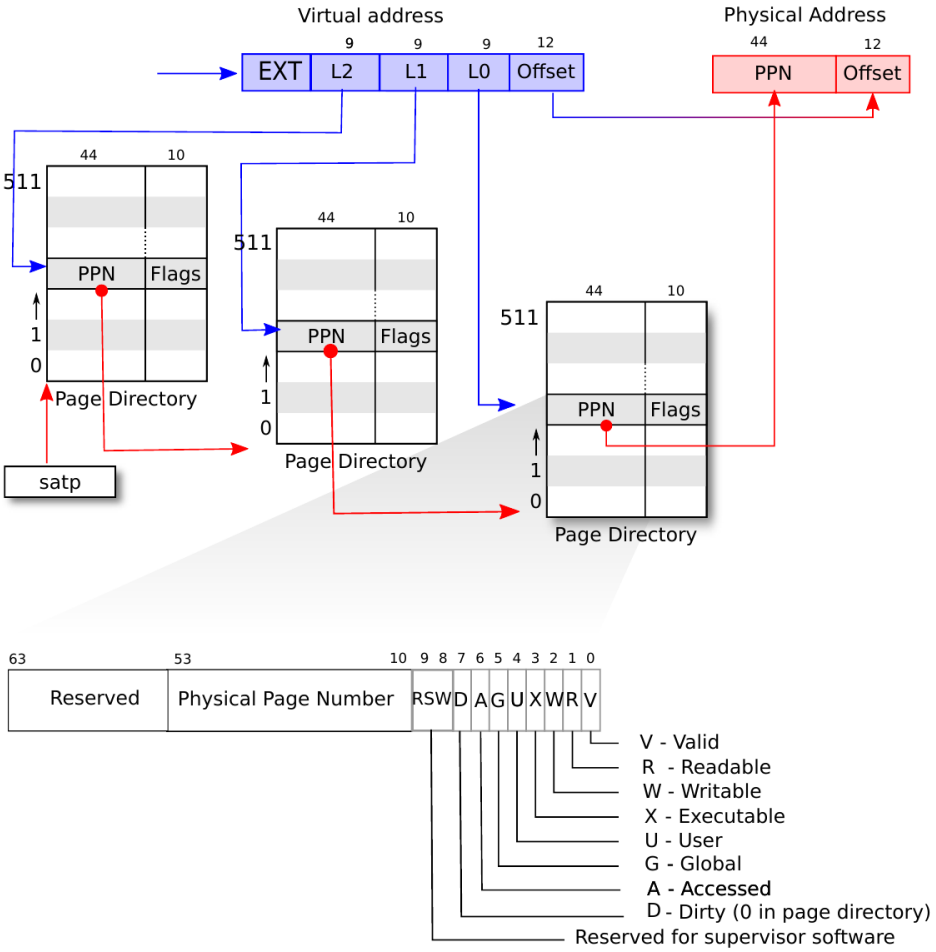
Trong Sv39 RISC-V, 25 bit cao của địa chỉ ảo không được dùng để dịch. Địa chỉ vật lý cũng có chỗ để phát triển thêm: định dạng PTE còn đủ chỗ để số trang vật lý lớn thêm 10 bit nữa. Các nhà thiết kế RISC-V chọn những con số này dựa trên dự đoán công nghệ. 2^{39} byte là 512 GB, đủ lớn cho không gian địa chỉ của các ứng dụng chạy trên máy tính RISC-V. 2^{56} là đủ không gian bộ nhớ vật lý trong tương lai gần để chứa nhiều thiết bị I/O và chip RAM. Nếu cần nhiều hơn, các nhà thiết kế RISC-V đã định nghĩa Sv48 với địa chỉ ảo 48 bit [3].

Như Hình 3.2 cho thấy, một CPU RISC-V dịch địa chỉ ảo sang địa chỉ vật lý qua ba bước. Một page table được lưu trong bộ nhớ vật lý dưới dạng một **cây ba cấp**. Gốc của cây là một "trang page-table" 4096 byte chứa 512 PTE; các PTE này chứa địa chỉ vật lý của các trang page-table ở cấp kế tiếp của cây. Mỗi trang trong số đó lại chứa 512 PTE cho cấp cuối cùng của cây. Phần cứng phân trang dùng 9 bit cao trong 27 bit để chọn một PTE trong trang page-table gốc, 9 bit giữa để chọn một PTE trong một trang page-table ở cấp kế tiếp, và 9 bit thấp để chọn PTE cuối cùng. (Trong Sv48 RISC-V, page table có bốn cấp, và các bit từ 39 đến 47 của địa chỉ ảo được dùng để đánh chỉ số vào cấp cao nhất.)

Nếu bất kỳ PTE nào trong ba PTE cần thiết để dịch một địa chỉ không hiện diện (not present), phần cứng phân trang sẽ phát sinh một **ngoại lệ lỗi trang** (page-fault exception), để kernel tự xử lý ngoại lệ đó (xem Chương 4).

Cấu trúc ba cấp ở Hình 3.2 cho phép ghi lại các PTE một cách tiết kiệm bộ nhớ so với thiết kế một cấp ở Hình 3.1. Trong trường hợp thường gặp là những dải địa chỉ ảo rất lớn không có ánh xạ nào, cấu trúc ba cấp có thể bỏ qua hoàn toàn cả các page directory. Ví dụ, nếu một ứng dụng chỉ dùng vài trang bắt đầu từ địa chỉ không, thì các mục từ 1 đến 511 của page directory cấp cao nhất là không hợp lệ, và kernel không phải cấp phát các trang cho 511 page directory trung gian đó. Hơn nữa, kernel cũng không phải cấp phát các trang cho các page directory cấp thấp nhất của 511 page directory trung gian kia. Vì vậy, trong ví dụ này, thiết kế ba cấp tiết kiệm được 511 trang cho các page directory trung gian và 511 × 512 trang cho các page directory cấp thấp nhất.

Mặc dù CPU duyệt cấu trúc ba cấp này bằng phần cứng như một phần của việc thực thi một lệnh load hoặc store, một nhược điểm tiềm tàng của ba cấp là CPU phải nạp ba PTE từ bộ nhớ để dịch địa chỉ ảo trong lệnh load/store thành địa chỉ vật lý. Để tránh chi phí nạp PTE từ bộ nhớ vật lý, một CPU RISC-V lưu đệm (cache) các page table entry trong một **Translation Look-aside Buffer (TLB)**.



Hình 3.2: Chi tiết việc dịch địa chỉ của RISC-V.

Giải thích hình – đọc Hình 3.2 (cây ba cấp và cấu trúc một PTE)

Hình này gồm hai phần: nửa trên là **cách phần cứng đi qua ba cấp**, nửa dưới là **bố cục 64 bit của một PTE**.

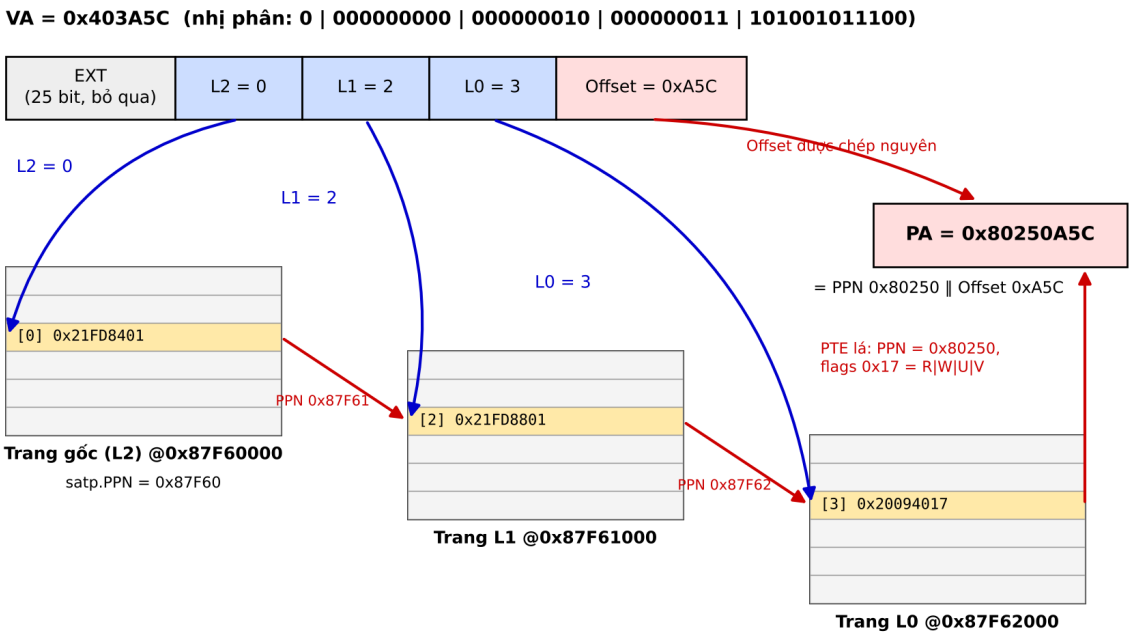
- **Địa chỉ ảo** (thanh xanh phía trên) giờ được tách thành **EXT | L2 | L1 | L0 | Offset** với độ rộng 25 | 9 | 9 | 9 | 12 bit. L2, L1, L0 là ba chỉ số, mỗi chỉ số 9 bit nên chọn được 512 mục (0…511) – vừa đúng bằng số PTE nằm gọn trong một trang 4096 byte (512 × 8 byte).
- **Thanh ghi satp** (hộp nhỏ bên trái) chứa địa chỉ trang gốc. Mũi tên đỏ từ satp tới page directory đầu tiên (cấp L2).
- Bước 1: L2 chọn một PTE trong page directory gốc. PPN trong PTE đó (mũi tên đỏ) trỏ tới page directory cấp giữa. Bước 2: L1 chọn PTE trong page directory cấp giữa, PPN của nó trỏ tới page directory cấp cuối. Bước 3: L0 chọn PTE **lá** (leaf) trong page directory cuối; PPN của PTE lá là 44 bit cao của địa chỉ vật lý (ô đỏ "Physical Address"). 12 bit Offset được chép thẳng xuống làm 12 bit thấp.
- **Nửa dưới – một PTE 64 bit:** bit 63–54 *Reserved* (dành riêng); bit 53–10 là **Physical Page Number** (44 bit); bit 9–8 là **RSW** (dành cho phần mềm supervisor, tức kernel, tự dùng); rồi các cờ: **D** (Dirty – bit 7), **A** (Accessed – bit 6), **G** (Global – bit 5), **U** (User – bit 4), **X** (Executable – bit 3), **W** (Writable – bit 2), **R** (Readable – bit 1), **V** (Valid – bit 0).

Ví dụ minh họa – ví dụ dịch địa chỉ ảo 0x403A5C bằng cây ba cấp (từng bước)

Giả sử một tiến trình đọc địa chỉ ảo `VA = 0x403A5C`. Phần cứng làm như sau:

1. **Tách bit.** `Offset = VA & 0xFFF = 0xA5C`. Số trang ảo `VPN = VA >> 12 = 0x403`. Từ đó `L0 = VPN & 0x1FF = 3`, `L1 = (VPN >> 9) & 0x1FF = 2`, `L2 = (VPN >> 18) & 0x1FF = 0`.
2. **Cấp L2.** Đọc `satp` → giả sử PPN gốc = `0x87F60`, tức trang gốc ở địa chỉ vật lý `0x87F60000`. Lấy PTE thứ L2 = 0 của trang này. Giả sử `PTE = 0x21FD8401`: cờ thấp là `0x01` (chỉ V=1, R=W=X=0 nghĩa là nó **trỏ tới cấp kế tiếp** chứ không phải trang dữ liệu), PPN = `PTE >> 10 = 0x87F61` → trang page-directory L1 ở `0x87F61000`.
3. **Cấp L1.** Lấy PTE thứ L1 = 2 trong trang `0x87F61000`; giả sử `PTE = 0x21FD8801` → PPN = `0x87F62` → trang L0 ở `0x87F62000`.
4. **Cấp L0.** Lấy PTE thứ L0 = 3 trong trang `0x87F62000`; giả sử `PTE = 0x20094017`. Cờ `0x17` = `0b10111` = V | R | W | U (một trang dữ liệu của user, đọc/ghi được). PPN = `0x20094017 >> 10 = 0x80250`.
5. **Ghép địa chỉ vật lý.** `PA = (0x80250 << 12) | 0xA5C = 0x80250A5C`. CPU gửi địa chỉ này tới RAM.

Chú ý: để dịch **một** lần truy cập bộ nhớ, CPU phải đọc **3 PTE** (3 lần truy cập RAM) rồi mới truy cập dữ liệu (lần thứ 4). Đó chính là lý do TLB tồn tại. Hình dưới mô tả lại ví dụ này.



Hình bổ sung (người dịch vẽ): ví dụ dịch địa chỉ ảo 0x403A5C qua ba cấp. Số liệu PTE là giả định để minh họa.

Ví dụ minh họa – ví dụ tiết kiệm bộ nhớ của cây ba cấp, tính bằng số

Giả sử một chương trình nhỏ chỉ dùng 3 trang ảo nằm liền nhau ở đầu không gian địa chỉ (địa chỉ 0x0000, 0x1000, 0x2000: một trang code, một trang data, một trang stack). Ba trang này có cùng L2 = 0 và L1 = 0, chỉ khác L0 (= 0, 1, 2). Do đó kernel chỉ cần:

- 1 trang gốc (L2) + 1 trang L1 + 1 trang L0 = **3 trang page-table = 12 KB**.
- So với bảng phẳng cần **1 GiB** (xem ví dụ phía trên) – chênh lệch khoảng 87.000 lần.

Các mục 1…511 của trang gốc để PTE_V = 0, nên 511 trang L1 và 511 × 512 trang L0 tương ứng **không bao giờ được cấp phát**. Đúng như con số 511 và 511 × 512 trong đoạn văn trên.

Giải thích thêm – TLB hoạt động như thế nào

TLB là một bộ nhớ đệm nhỏ, rất nhanh, nằm ngay trong CPU, lưu các cặp "số trang ảo → PTE (PPN + cờ)" vừa dùng gần đây. Khi CPU cần dịch địa chỉ ảo, nó tra TLB trước:

- **TLB hit** (có sẵn): lấy ngay PPN và cờ, **không** phải đọc 3 PTE từ RAM.
- **TLB miss** (không có): phần cứng đi qua cây ba cấp (3 lần đọc RAM), rồi nạp kết quả vào TLB cho lần sau.

Vì chương trình có tính cục bộ (liên tục truy cập các trang gần nhau), tỉ lệ hit thường rất cao, nên ba cấp gần như không làm chậm. Nhưng TLB là một bản sao: nếu kernel đổi page table trong RAM mà không báo cho CPU thì TLB có thể còn giữ ánh xạ cũ – xem lệnh `sfence.vma` ở mục 3.3.

Mỗi PTE chứa các bit cờ cho phần cứng phân trang biết địa chỉ ảo tương ứng được phép sử dụng như thế nào. `PTE_V` cho biết PTE có hiện diện hay không: nếu không được đặt, một tham chiếu tới trang sẽ gây ra ngoại lệ (tức là không được phép). `PTE_R` điều khiển việc các lệnh có được phép đọc trang hay không. `PTE_W` điều khiển việc các lệnh có được phép ghi vào trang hay không. `PTE_X` điều khiển việc CPU có thể diễn giải nội dung của trang như các lệnh và thực thi chúng hay không. `PTE_U` điều khiển việc các lệnh ở chế độ user có được phép truy cập trang hay không; nếu `PTE_U` không được đặt, PTE chỉ có thể được dùng ở chế độ supervisor. Hình 3.2 cho thấy toàn bộ cơ chế này hoạt động ra sao. Các cờ và mọi cấu trúc khác liên quan đến phần cứng trang đều được định nghĩa trong (kernel/riscv.h).

Giải thích thêm – các cờ PTE và cách đọc/ghi chúng

Cờ	Bit	Giá trị	Ý nghĩa
`PTE_V`	0	1<<0 = 0x01	PTE hợp lệ. Nếu = 0, mọi truy cập đều gây page fault.
`PTE_R`	1	1<<1 = 0x02	Cho phép đọc (load).
`PTE_W`	2	1<<2 = 0x04	Cho phép ghi (store).
`PTE_X`	3	1<<3 = 0x08	Cho phép nạp lệnh để thực thi.
`PTE_U`	4	1<<4 = 0x10	Cho phép chế độ user truy cập (nếu = 0: chỉ kernel).

Các tổ hợp thường gặp trong xv6: trang code của user = `PTE_R|PTE_X|PTE_U|PTE_V` = 0x1B; trang data/stack/heap của user = `PTE_R|PTE_W|PTE_U|PTE_V` = 0x17; trang code của kernel = `PTE_R|PTE_X|PTE_V` = 0x0B; trang dữ liệu của kernel = `PTE_R|PTE_W|PTE_V` = 0x07.

Quy ước của RISC-V: nếu một PTE hợp lệ mà **cả R, W, X đều bằng 0** thì nó là PTE **trỏ tới cấp kế tiếp** (như `0x21FD8401` ở ví dụ trên); nếu có ít nhất một trong R/W/X thì nó là PTE **lá** (trỏ tới trang dữ liệu). Phần cứng cũng tự bật bit A (Accessed) khi trang được truy cập và bit D (Dirty) khi trang bị ghi; kernel có thể dùng chúng để biết trang nào đã dùng/đã sửa (xv6 không dùng). Bit PTE_U có tác dụng cô lập: kernel có thể ánh xạ trang của mình vào cùng page table với user mà user vẫn không truy cập được.

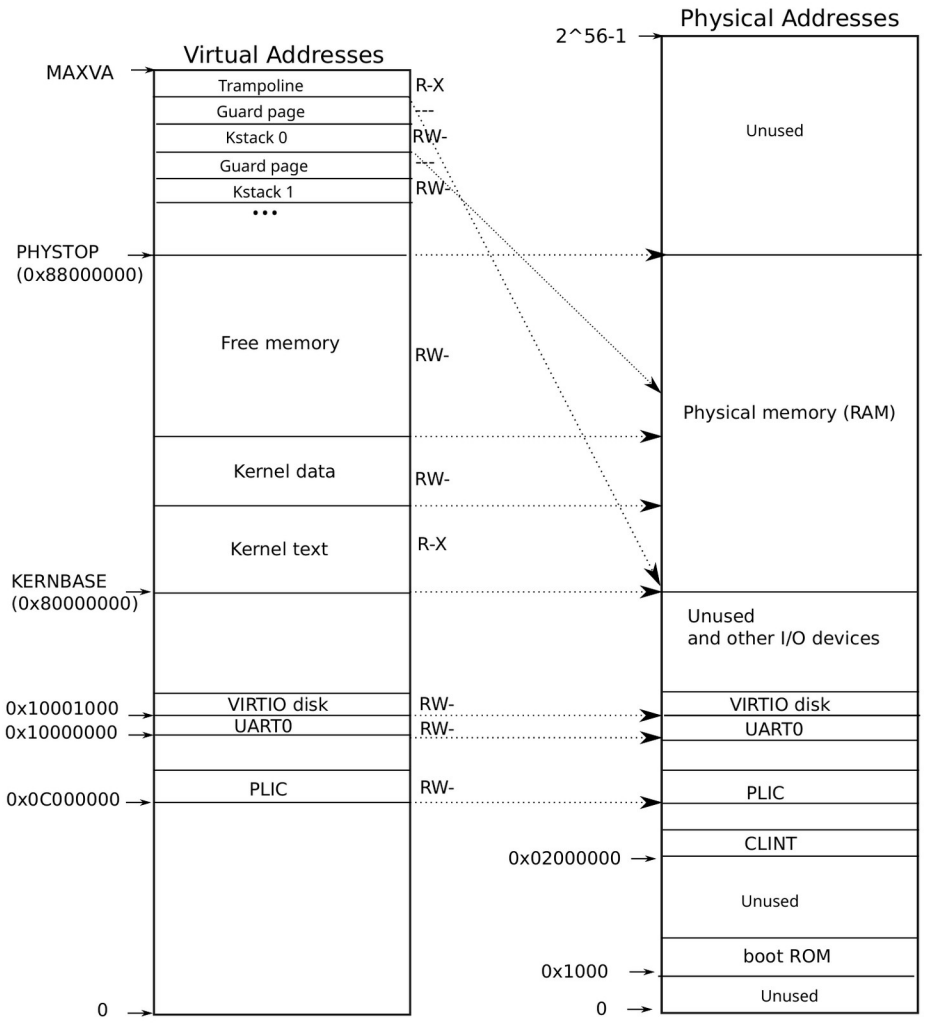
Các hằng/macro hữu ích trong kernel/riscv.h: `PA2PTE(pa) = (pa >> 12) << 10` (đặt địa chỉ vật lý vào trường PPN), `PTE2PA(pte) = (pte >> 10) << 12` (lấy lại địa chỉ vật lý), `PX(level, va) = (va >> (12 + 9*level)) & 0x1FF` (lấy chỉ số 9 bit của cấp `level`).

Để bảo một CPU dùng một page table, kernel phải ghi địa chỉ vật lý của trang page-table gốc vào thanh ghi `satp`. CPU sẽ dịch mọi địa chỉ sinh ra bởi các lệnh tiếp theo bằng page table mà `satp` của chính nó trỏ tới. Mỗi CPU có `satp` riêng để các CPU khác nhau có thể chạy các tiến trình khác nhau, mỗi tiến trình có một không gian địa chỉ riêng được mô tả bởi page table của nó.

> **Giải thích thêm – thanh ghi satp**
>
> `satp` (Supervisor Address Translation and Protection) gồm 3 phần: **MODE** (bit 63–60) chọn kiểu dịch địa chỉ (8 = Sv39, 0 = tắt phân trang), **ASID** (bit 59–44, định danh không gian địa chỉ, xv6 không dùng) và **PPN** (bit 43–0, số trang vật lý của trang gốc). Trong xv6: `MAKE_SATP(pagetable) = (8L << 60) | (pagetable >> 12)`. Ví dụ, trang gốc ở `0x87F60000` thì `satp = 0x8000000000087F60`. Ghi vào `satp` là lệnh duy nhất cần thiết để "đổi cả thế giới địa chỉ" của một CPU – đây là điều xảy ra khi hệ điều hành chuyển từ tiến trình này sang tiến trình khác.

Từ góc nhìn của kernel, một page table là dữ liệu được lưu trong bộ nhớ, và kernel tạo và sửa đổi page table bằng mã rất giống với mã bạn có thể thấy cho bất kỳ cấu trúc dữ liệu hình cây nào.

Một vài ghi chú về các thuật ngữ dùng trong cuốn sách này. **Bộ nhớ vật lý** (physical memory) là các ô lưu trữ trong RAM. Một byte bộ nhớ vật lý có một địa chỉ, gọi là **địa chỉ vật lý**. Các lệnh dereference địa chỉ (như load, store, jump và gọi hàm) chỉ dùng địa chỉ ảo, mà phần cứng phân trang dịch thành địa chỉ vật lý rồi gửi đến phần cứng RAM để đọc hoặc ghi. Một **không gian địa chỉ** (address space) là tập các địa chỉ ảo hợp lệ trong một page table cho trước; mỗi tiến trình xv6 có một không gian địa chỉ user riêng, và kernel xv6 cũng có không gian địa chỉ của riêng nó. **Bộ nhớ user** (user memory) là không gian địa chỉ user của một tiến trình cộng với bộ nhớ vật lý mà page table cho phép tiến trình truy cập. **Bộ nhớ ảo** (virtual memory) là các ý tưởng và kỹ thuật liên quan đến việc quản lý page table và sử dụng chúng để đạt các mục tiêu như cô lập.



Hình 3.3: Bên trái: không gian địa chỉ kernel của xv6. RWX là các quyền đọc, ghi và thực thi của PTE. Bên phải: không gian địa chỉ vật lý RISC-V mà xv6 kỳ vọng thấy.

Giải thích hình – đọc Hình 3.3 (bố cục bộ nhớ của kernel)

Hình có hai cột. **Cột trái** là không gian địa chỉ **ảo** của kernel (tăng dần từ 0 ở dưới lên MAXVA ở trên); **cột phải** là không gian địa chỉ **vật lý** của máy QEMU/RISC-V (từ 0 đến $2^{56}-1$). Các mũi tên chấm chấm nối từng vùng ảo tới vùng vật lý mà nó ánh xạ; nhãn cạnh mỗi vùng (R-X, RW-, ---) là quyền R/W/X của PTE.

- **Các thiết bị (phần thấp, dưới 0x80000000):** PLIC ở 0x0C000000, UART0 ở 0x10000000, VIRTIO disk ở 0x10001000 – ánh xạ **trực tiếp** (ảo = vật lý), quyền RW-. Bên phải còn có CLINT (0x02000000) và boot ROM (0x1000) nhưng xv6 không ánh xạ chúng trong kernel page table của hình này (vùng "unused").
- **RAM từ KERNBASE = 0x80000000 đến PHYSTOP = 0x88000000** (128 MB): gồm **Kernel text** (R-X: code kernel), **Kernel data** (RW-) và **Free memory** (RW-: các trang để cấp phát cho user, page table, ...). Tất cả cũng ánh xạ trực tiếp.
- **Phía trên cùng (gần MAXVA)** là các vùng **không** ánh xạ trực tiếp: **Trampoline** (R-X) ở trang cao nhất, rồi lần lượt các cặp **Guard page** (---, PTE không hợp lệ) và **Kstack 0, Kstack 1, ...** (RW-). Hai mũi tên chấm chéo dài từ Trampoline và Kstack đi xuống RAM cho thấy: chúng ánh xạ tới các trang vật lý nằm trong vùng RAM bình thường (trampoline chính là code nằm trong kernel text; stack là các trang lấy từ vùng RAM), tức **cùng một trang vật lý có thể được truy cập qua hai địa chỉ ảo**.

3.2 Không gian địa chỉ kernel (Kernel address space)

Xv6 duy trì một page table cho mỗi tiến trình, mô tả không gian địa chỉ user của tiến trình đó, cộng với một page table duy nhất mô tả không gian địa chỉ của kernel. Kernel cấu hình bố cục không gian địa chỉ của mình để nó có thể truy cập bộ nhớ vật lý và các tài nguyên phần cứng khác nhau tại các địa chỉ ảo có thể đoán trước. Hình 3.3 cho thấy bố cục này ánh xạ địa chỉ ảo của kernel sang địa chỉ vật lý như thế nào. Tệp (kernel/memlayout.h) khai báo các hằng số cho bố cục bộ nhớ kernel của xv6.

QEMU mô phỏng một máy tính có RAM (bộ nhớ vật lý) bắt đầu tại địa chỉ vật lý 0x80000000 và kéo dài ít nhất đến 0x88000000, mà xv6 gọi là `PHYSTOP`. Mô phỏng QEMU cũng bao gồm các thiết bị I/O như giao diện đĩa. QEMU phơi bày giao diện thiết bị cho phần mềm dưới dạng các thanh ghi điều khiển ánh xạ vào bộ nhớ (memory-mapped control registers) nằm dưới 0x80000000 trong không gian địa chỉ vật lý. Kernel có thể tương tác với thiết bị bằng cách đọc/ghi các địa chỉ vật lý đặc biệt này; những lần đọc và ghi như vậy giao tiếp với phần cứng thiết bị chứ không phải với RAM. Chương 4 giải thích cách xv6 tương tác với thiết bị.

Giải thích thêm – memory-mapped I/O (MMIO)

Với thiết bị như UART (cổng nối tiếp), phần cứng gán cho nó một dải địa chỉ vật lý (từ 0x10000000). Một lệnh store tới 0x10000000 **không** ghi vào RAM mà gửi byte đó tới thiết bị UART – ví dụ `*(volatile char*)0x10000000 = 'A'` làm ký tự 'A' hiện ra console. Ưu điểm: không cần lệnh I/O đặc biệt, chỉ dùng load/store thông thường. Vì vậy kernel phải ánh xạ các địa chỉ này vào page table của mình (với quyền RW, không có X) thì mới điều khiển được thiết bị.

Kernel truy cập RAM và các thanh ghi thiết bị ánh xạ bộ nhớ bằng "ánh xạ trực tiếp" (direct mapping); nghĩa là ánh xạ các tài nguyên tại các địa chỉ ảo **bằng** địa chỉ vật lý. Ví dụ, bản thân kernel nằm tại KERNBASE=0x80000000 trong cả không gian địa chỉ ảo lẫn bộ nhớ vật lý. Ánh xạ trực tiếp đơn giản hóa mã kernel đọc hoặc ghi bộ nhớ vật lý. Ví dụ, khi `fork` cấp phát bộ nhớ user cho tiến trình con, bộ cấp phát trả về địa chỉ vật lý của bộ nhớ đó; `fork` dùng địa chỉ đó trực tiếp như một địa chỉ ảo khi sao chép bộ nhớ user của cha sang con.

Ví dụ minh họa – ánh xạ trực tiếp (direct map) để làm gì?

Giả sử `kalloc()` trả về địa chỉ vật lý `0x87FF9000`. Kernel muốn xóa trang này về 0: chỉ cần `memset((void*)0x87FF9000, 0, 4096)`. Vì page table của kernel ánh xạ ảo `0x87FF9000` → vật lý `0x87FF9000`, lệnh này ghi đúng trang đó mà không cần bất kỳ bước đổi địa chỉ nào. Nếu không có ánh xạ trực tiếp, kernel phải tự dựng một ánh xạ tạm cho mỗi trang vật lý muốn đụng tới – rất phiền phức. Cũng vì thế mà hàm `walk` ở mục 3.3 có thể lấy địa chỉ vật lý trong PTE rồi dùng nó như con trỏ để đọc tiếp cấp dưới.

Có một vài địa chỉ ảo của kernel không được ánh xạ trực tiếp:

- **Trang trampoline.** Nó được ánh xạ ở đỉnh không gian địa chỉ ảo; các page table của user cũng có cùng ánh xạ này. Chương 4 thảo luận vai trò của trang trampoline, nhưng ở đây ta thấy một trường hợp sử dụng thú vị của page table: một trang vật lý (chứa mã trampoline) được ánh xạ hai lần trong không gian địa chỉ ảo của kernel: một lần ở đỉnh không gian địa chỉ ảo và một lần với ánh xạ trực tiếp.
- **Các trang kernel stack.** Mỗi tiến trình có một kernel stack riêng, được ánh xạ ở vùng cao để bên dưới nó xv6 có thể để lại một trang canh gác (guard page) không được ánh xạ. PTE của guard page không hợp lệ (tức là `PTE_V` không được đặt), để nếu kernel làm tràn một kernel stack, nhiều khả năng nó sẽ gây ra một ngoại lệ và kernel sẽ panic. Nếu không có guard page, một stack bị tràn sẽ ghi đè lên bộ nhớ kernel khác, dẫn đến hoạt động sai. Một vụ crash kiểu panic thì tốt hơn.

Mặc dù kernel dùng các stack của nó qua các ánh xạ ở vùng bộ nhớ cao, chúng cũng truy cập được với kernel qua một địa chỉ ánh xạ trực tiếp. Một thiết kế thay thế có thể chỉ dùng ánh xạ trực tiếp, và dùng các stack tại địa chỉ ánh xạ trực tiếp. Tuy nhiên, trong cách bố trí đó, việc tạo guard page sẽ đòi hỏi hủy ánh xạ các địa chỉ ảo mà nếu không thì vẫn tham chiếu tới bộ nhớ vật lý, và bộ nhớ đó sau này sẽ khó sử dụng.

Ví dụ minh họa – vị trí các kernel stack và guard page

Trong kernel/memlayout.h, `TRAMPOLINE = MAXVA − PGSIZE` và `KSTACK(p) = TRAMPOLINE − (p+1) × 2 × PGSIZE`, với `MAXVA = 2^{38} = 0x4000000000` và `PGSIZE = 0x1000`. Mỗi tiến trình chiếm **2 trang ảo** liên tiếp: 1 trang stack (hợp lệ) và 1 trang guard (không hợp lệ). Với tiến trình số 0 và 1:

Địa chỉ ảo	Trang	PTE_V
0x3FFFFFF000	Trampoline (R-X)	1
0x3FFFFFE000	Guard page	0
0x3FFFFFD000	Kstack của tiến trình 0 (RW-)	1
0x3FFFFFC000	Guard page	0
0x3FFFFFB000	Kstack của tiến trình 1 (RW-)	1

Stack mọc **xuống** (từ địa chỉ cao về thấp). Nếu kstack 0 tràn xuống dưới 0x3FFFFFD000, lệnh store đầu tiên vào 0x3FFFFFCFF8 chạm vào guard page (V=0) → page fault → kernel panic ngay, thay vì lặng lẽ ghi đè lên kstack của tiến trình 1. Chi phí: mỗi stack "tốn" thêm một trang địa chỉ ảo, nhưng trang guard **không tốn trang vật lý nào**, vì nó không được ánh xạ.

Kernel ánh xạ các trang của trampoline và của kernel text với quyền `PTE_R` và `PTE_X`. Kernel đọc và thực thi các lệnh từ những trang này. Kernel ánh xạ các trang còn lại với quyền `PTE_R` và `PTE_W`, để nó có thể đọc và ghi bộ nhớ trong các trang đó. Các ánh xạ cho guard page là không hợp lệ.

3.3 Mã: tạo một không gian địa chỉ (Code: creating an address space)

Phần lớn mã xv6 thao tác trên không gian địa chỉ và page table nằm trong `vm.c` (kernel/vm.c:1). Cấu trúc dữ liệu trung tâm là `pagetable_t`, thực chất là một con trỏ tới một trang page-table gốc của RISC-V; một `pagetable_t` có thể là page table của kernel, hoặc một trong các page table của từng tiến trình. Các hàm trung tâm là `walk`, tìm PTE cho một địa chỉ ảo, và `mappages`, cài đặt các PTE cho những ánh xạ mới. Các hàm bắt đầu bằng `kvm` thao tác trên page table của kernel; các hàm bắt đầu bằng `uvm` thao tác trên page table của user; các hàm khác dùng cho cả hai. `copyout` và `copyin` sao chép dữ liệu tới và từ các địa chỉ ảo của user được cung cấp làm đối số của system call; chúng nằm trong `vm.c` vì chúng cần dịch tường minh các địa chỉ đó để tìm bộ nhớ vật lý tương ứng.

Giải thích thêm – sơ đồ các hàm trong vm.c và quan hệ gọi nhau

Hàm	Vai trò
`walk(pt, va, alloc)`	Tìm (và nếu alloc ≠ 0 thì tạo) PTE cấp lá ứng với `va`; trả về **con trỏ tới PTE**.
`mappages(pt, va, size, pa, perm)`	Với mỗi trang trong [va, va+size): gọi walk rồi ghi PTE = PA2PTE(pa) | perm | PTE_V.
`kvmmake / kvminit`	Dựng page table của kernel (gọi `kvmmap` → `mappages`).
`kvminithart`	Ghi `satp` để CPU hiện tại dùng page table kernel, rồi `sfence.vma`.
`uvmalloc / uvmdealloc`	Mở rộng / thu nhỏ bộ nhớ user (dùng `kalloc`, `mappages`, `uvmunmap`).
`uvmunmap`	Gỡ ánh xạ (và tùy chọn `kfree` trang vật lý).
`walkaddr`	Dịch một địa chỉ ảo của user thành địa chỉ vật lý (trả 0 nếu không hợp lệ hoặc không có PTE_U).
`copyin / copyout`	Chép dữ liệu từ / tới địa chỉ ảo của user, dùng `walkaddr` để dịch từng trang.

Ngay từ sớm trong quá trình khởi động, `main` gọi `kvminit` (kernel/vm.c:54) để tạo page table của kernel bằng `kvmmake` (kernel/vm.c:20). Lời gọi này xảy ra trước khi xv6 bật phân trang trên RISC-V, nên các địa chỉ lúc này tham chiếu trực tiếp tới bộ nhớ vật lý. `kvmmake` trước tiên cấp phát một trang bộ nhớ vật lý để chứa trang page-table gốc. Sau đó nó gọi `kvmmap` để cài đặt các phép dịch mà kernel cần. Các phép dịch này bao gồm lệnh và dữ liệu của kernel, bộ nhớ vật lý đến `PHYSTOP`, và các dải bộ nhớ thực chất là thiết bị. `proc_mapstacks` (kernel/proc.c:33) cấp phát một kernel stack cho mỗi tiến trình. Nó gọi `kvmmap` để ánh xạ từng stack tại địa chỉ ảo do `KSTACK` sinh ra, để chừa chỗ cho các trang canh gác stack không hợp lệ.

Ví dụ minh họa – kvmmake dựng những ánh xạ nào (khớp với Hình 3.3)

Vùng	Địa chỉ ảo → vật lý	Quyền
UART0	0x10000000 → 0x10000000	R W
VIRTIO disk	0x10001000 → 0x10001000	R W
PLIC	0x0C000000 → 0x0C000000 (kích thước 0x400000)	R W
Kernel text	KERNBASE → KERNBASE (đến hết `etext`)	R X
Kernel data + RAM còn lại	etext → etext (đến PHYSTOP)	R W
Trampoline	TRAMPOLINE (đỉnh) → vật lý của `trampoline` (trong kernel text)	R X
Kernel stacks	KSTACK(p) → trang `kalloc` riêng cho từng tiến trình p	R W

Thứ tự trong `kvmmake`: `kalloc` trang gốc → `memset` về 0 (mọi PTE ban đầu có V = 0) → lần lượt `kvmmap` các vùng trên → `proc_mapstacks`. Lưu ý khi mã này chạy, CPU còn chưa bật phân trang, nên các con trỏ ở đây là địa chỉ vật lý thật.

`kvmmap` (kernel/vm.c:132) gọi `mappages` (kernel/vm.c:144), hàm cài đặt các ánh xạ vào một page table cho một dải địa chỉ ảo tới một dải địa chỉ vật lý tương ứng. Nó làm việc này riêng cho từng địa chỉ ảo trong dải, theo từng khoảng bằng một trang. Với mỗi địa chỉ ảo cần ánh xạ, `mappages` gọi `walk` để tìm địa chỉ của PTE cho địa chỉ đó. Sau đó nó khởi tạo PTE để chứa số trang vật lý liên quan, các quyền mong muốn (`PTE_W`, `PTE_X`, và/hoặc `PTE_R`), và `PTE_V` để đánh dấu PTE là hợp lệ (kernel/vm.c:165).

Giải thích thêm – thuật toán mappages (giả mã) và ví dụ

```
mappages(pagetable, va, size, pa, perm):
    a    = PGROUNDDOWN(va)              # làm tròn xuống đầu trang
    last = PGROUNDDOWN(va + size - 1)   # trang cuối cùng cần ánh xạ
    lặp:
        pte = walk(pagetable, a, alloc=1)   # tìm PTE lá (tạo cấp trung gian
nếu thiếu)
        nếu pte == NULL: trả về lỗi (hết bộ nhớ)
        nếu *pte có PTE_V: panic("remap")   # đã ánh xạ rồi -> lỗi lập trình
        *pte = PA2PTE(pa) | perm | PTE_V
        nếu a == last: dừng
        a += 4096;  pa += 4096
```

Ví dụ: `mappages(kpgtbl, 0x10000000, 0x1000, 0x10000000, PTE_R|PTE_W)` (ánh xạ UART0): `a = last = 0x10000000` nên vòng lặp chạy **đúng 1 lần**; PTE lá được ghi `PA2PTE(0x10000000) | 0x06 | 0x01 = (0x10000 << 10) | 0x07 = 0x4000007`. Còn nếu ánh xạ 3 trang liên tiếp từ `0x80000000`, vòng lặp chạy 3 lần với (a, pa) = (0x80000000, 0x80000000), (0x80001000, 0x80001000), (0x80002000, 0x80002000).

`walk` (kernel/vm.c:86) mô phỏng phần cứng phân trang RISC-V khi nó tra cứu PTE cho một địa chỉ ảo (xem Hình 3.2). `walk` đi xuống page table từng cấp một, dùng 9 bit địa chỉ ảo của mỗi cấp để đánh chỉ số vào trang page directory liên quan. Ở mỗi cấp nó tìm thấy hoặc PTE của trang page directory cấp kế tiếp, hoặc PTE của trang cuối cùng (kernel/vm.c:92). Nếu một PTE trong trang page directory cấp một hoặc cấp hai không hợp lệ, thì trang directory cần thiết chưa được cấp phát; nếu đối số `alloc` được đặt, `walk` cấp phát một trang page-table mới và đặt địa chỉ vật lý của nó vào PTE. Nó trả về địa chỉ của PTE ở tầng thấp nhất trong cây (kernel/vm.c:102).

Giải thích thêm – thuật toán walk (giả mã) và ví dụ chạy từng bước

```
walk(pagetable, va, alloc):
    for level = 2 down to 1:                  # chỉ đi qua 2 cấp trên
        pte = &pagetable[PX(level, va)]       # PX = 9 bit của cấp 'level'
        nếu *pte & PTE_V:                     # cấp dưới đã tồn tại
            pagetable = PTE2PA(*pte)          # lấy địa chỉ (vật lý = ảo nhờ
direct map)
        ngược lại:                            # cấp dưới chưa có
            nếu !alloc hoặc kalloc() thất bại: trả về NULL
            pagetable = trang mới (đã xóa về 0)
            *pte = PA2PTE(pagetable) | PTE_V # nối trang mới vào cây
    trả về &pagetable[PX(0, va)]              # địa chỉ của PTE lá (có thể chưa
hợp lệ)
```

Ví dụ. Tiến trình mới có cây rỗng (chỉ có trang gốc, toàn 0). Gọi `walk(pt, 0x403A5C, alloc=1)` với `L2 = 0, L1 = 2, L0 = 3`:

1. level 2: `pte = &gốc[0]`, V = 0 → `kalloc` trang A, xóa 0, ghi `gốc[0] = PA2PTE(A) | V`; chuyển sang A.
2. level 1: `pte = &A[2]`, V = 0 → `kalloc` trang B, ghi `A[2] = PA2PTE(B) | V`; chuyển sang B.
3. Trả về `&B[3]`. Lúc này PTE lá này vẫn có V = 0; **`mappages` mới là nơi ghi** `*pte = PA2PTE(pa) | perm | PTE_V`.

Gọi `walk` lần nữa cho `0x404000` (L2 = 0, L1 = 2, L0 = 4): cả hai cấp trên đã có V = 1 nên **không cấp phát thêm** trang nào, chỉ trả về `&B[4]`. Còn nếu gọi với `alloc = 0` cho một địa chỉ mà cấp trên chưa tồn tại, `walk` trả về NULL – cách `walkaddr`/`uvmunmap` kiểm tra "địa chỉ này đã được ánh xạ chưa".

Đoạn mã trên phụ thuộc vào việc bộ nhớ vật lý được ánh xạ trực tiếp vào không gian địa chỉ ảo của kernel. Ví dụ, khi `walk` đi xuống các cấp của page table, nó lấy địa chỉ (vật lý) của page table cấp thấp hơn kế tiếp từ một PTE (kernel/vm.c:94), rồi dùng địa chỉ đó như một địa chỉ ảo để nạp PTE ở cấp thấp hơn kế tiếp (kernel/vm.c:92).

`main` gọi `kvminithart` (kernel/vm.c:62) để cài đặt page table của kernel. Nó ghi địa chỉ vật lý của trang page-table gốc vào thanh ghi `satp`. Sau đó CPU sẽ dịch địa chỉ bằng page table của kernel. Vì kernel dùng ánh xạ trực tiếp, địa chỉ (giờ là địa chỉ ảo) của lệnh kế tiếp sẽ ánh xạ tới đúng địa chỉ bộ nhớ vật lý.

Giải thích thêm – vì sao lệnh ngay sau khi ghi satp vẫn chạy đúng?

Ngay trước khi ghi `satp`, CPU đang chạy ở địa chỉ (vật lý) chẳng hạn `0x80001234`. Ngay sau khi phân trang được bật, PC vẫn bằng `0x80001238` nhưng giờ nó được hiểu là **địa chỉ ảo**, và page table kernel ánh xạ `0x80001238` → `0x80001238` (direct map). Nhờ đó dòng lệnh tiếp theo vẫn được nạp từ đúng chỗ và chương trình chạy liên tục. Nếu kernel dùng ánh xạ không trực tiếp, trước khi ghi `satp` ta phải đảm bảo vùng code đang chạy có mặt ở cả hai kiểu địa chỉ – rắc rối hơn nhiều.

Mỗi CPU RISC-V lưu đệm các page table entry trong một Translation Look-aside Buffer (TLB), và khi xv6 thay đổi một page table, nó phải bảo CPU vô hiệu hóa (invalidate) các mục TLB được lưu đệm tương ứng. Nếu không làm vậy, thì vào một lúc nào đó về sau TLB có thể dùng một ánh xạ cũ đã lưu đệm, trỏ tới một trang vật lý mà trong lúc đó đã được cấp phát cho một tiến trình khác, và kết quả là một tiến trình có thể ghi bậy lên bộ nhớ của tiến trình khác. RISC-V có một lệnh `sfence.vma` xả (flush) TLB của CPU hiện tại. Xv6 thực thi `sfence.vma` trong `kvminithart` sau khi nạp lại thanh ghi `satp`, và trong mã trampoline chuyển sang page table của user trước khi quay về không gian user (kernel/trampoline.S:89).

Ví dụ minh họa – kịch bản lỗi nếu quên sfence.vma

1. Tiến trình A có trang ảo `0x5000` ánh xạ tới trang vật lý `P = 0x87FF9000`. CPU đã đọc `0x5000` nên TLB lưu "0x5 → P (RW)".
2. Kernel gọi `uvmunmap` để thu hồi trang đó: xóa PTE và `kfree(P)`; P quay về free list. Nếu quên `sfence.vma`, TLB của CPU vẫn còn mục "0x5 → P".
3. Kernel `kalloc()` cấp đúng trang P cho tiến trình B để chứa dữ liệu bí mật (ví dụ mật khẩu).
4. Tiến trình A (hoặc lỗi trong A) ghi/đọc `0x5000`: TLB vẫn dịch thành P → A đọc/ghi được **bộ nhớ của B**. Đó là lỗ hổng cô lập nghiêm trọng, nên mọi thay đổi page table đều phải đi kèm `sfence.vma`.

Cũng cần phát lệnh `sfence.vma` trước khi thay đổi `satp`, để chờ cho tất cả các lệnh load và store đang chờ xử lý hoàn tất. Việc chờ này đảm bảo các cập nhật page table trước đó đã hoàn thành, và đảm bảo các lệnh load và store trước đó dùng page table cũ, chứ không phải cái mới.

Để tránh phải xả toàn bộ TLB, các CPU RISC-V có thể hỗ trợ các định danh không gian địa chỉ (address space identifiers – ASID) [3]. Khi đó kernel có thể chỉ xả các mục TLB của một không gian địa chỉ cụ thể. Xv6 không dùng tính năng này.

> **Giải thích thêm – ASID giúp gì?**
>
> Mỗi mục TLB được gắn thêm một nhãn ASID (lấy từ trường ASID trong `satp`). Khi chuyển tiến trình, kernel chỉ cần đổi `satp` sang page table (và ASID) của tiến trình mới; các mục TLB của tiến trình cũ vẫn nằm đó nhưng **không khớp** ASID nên không bị dùng, và có thể còn dùng lại nếu tiến trình cũ quay lại. Nhờ vậy tránh được việc xả sạch TLB ở mọi lần chuyển ngữ cảnh. Xv6 chọn đơn giản: không dùng ASID và xả TLB (khi cần).

3.4 Cấp phát bộ nhớ vật lý (Physical memory allocation)

Kernel phải cấp phát và giải phóng bộ nhớ vật lý tại thời gian chạy cho các page table, bộ nhớ user, kernel stack và bộ đệm pipe (pipe buffer).

Xv6 dùng bộ nhớ vật lý giữa phần cuối của kernel và `PHYSTOP` để cấp phát tại thời gian chạy. Nó cấp phát và giải phóng nguyên từng trang 4096 byte một lần. Nó theo dõi trang nào còn rảnh bằng cách xâu một danh sách liên kết xuyên qua chính các trang đó. Việc cấp phát là lấy một trang ra khỏi danh sách liên kết; việc giải phóng là thêm trang vừa giải phóng vào danh sách.

3.5 Mã: Bộ cấp phát bộ nhớ vật lý (Code: Physical memory allocator)

Bộ cấp phát nằm trong `kalloc.c` (kernel/kalloc.c:1). Cấu trúc dữ liệu của bộ cấp phát là một **danh sách rảnh** (free list) gồm các trang bộ nhớ vật lý có thể cấp phát. Phần tử danh sách của mỗi trang rảnh là một `struct run` (kernel/kalloc.c:17). Bộ cấp phát lấy bộ nhớ ở đâu để chứa cấu trúc dữ liệu này? Nó lưu cấu trúc `run` của mỗi trang rảnh ngay trong chính trang rảnh đó, vì không có gì khác được lưu ở đó. Danh sách rảnh được bảo vệ bởi một khóa quay (spin lock) (kernel/kalloc.c:21-24). Danh sách và khóa được gói trong một struct để làm rõ rằng khóa bảo vệ các trường trong struct. Tạm thời hãy bỏ qua khóa và các lời gọi `acquire` và `release`; Chương 6 sẽ xem xét việc khóa chi tiết.

> **Giải thích thêm – free list "luồn" qua chính các trang rảnh**
>
> Đây là mẹo quan trọng: bình thường, để quản lý N trang rảnh ta cần một mảng/danh sách N phần tử ở đâu đó – vậy thì ta cần cấp phát bộ nhớ để quản lý bộ nhớ! Xv6 tránh chuyện này vì một trang đang rảnh thì nội dung của nó vô nghĩa, nên 8 byte đầu của trang có thể dùng làm con trỏ `next` trỏ tới trang rảnh kế tiếp (`struct run { struct run *next; }`). Biến `kmem.freelist` chỉ giữ địa chỉ của trang đầu danh sách. Khi trang được cấp đi, nó là của người dùng và họ thoải mái ghi đè con trỏ `next` – không sao, vì trang đó không còn nằm trong danh sách. Chi phí quản lý chỉ là **8 byte tạm thời trong chính trang rảnh**, không tốn thêm bộ nhớ nào.

Hàm `main` gọi `kinit` để khởi tạo bộ cấp phát (kernel/kalloc.c:27). `kinit` khởi tạo danh sách rảnh để chứa mọi trang giữa phần cuối của kernel và `PHYSTOP`. Xv6 lẽ ra nên xác định lượng bộ nhớ vật lý khả dụng bằng cách phân tích thông tin cấu hình do phần cứng cung cấp. Thay

vào đó, xv6 giả định máy có 128 megabyte RAM. `kinit` gọi `freerange` để thêm bộ nhớ vào danh sách rảnh thông qua các lời gọi `kfree` theo từng trang. Một PTE chỉ có thể tham chiếu tới một địa chỉ vật lý được căn chỉnh theo ranh giới 4096 byte (là bội số của 4096), nên `freerange` dùng `PGROUNDUP` để đảm bảo nó chỉ giải phóng các địa chỉ vật lý đã căn chỉnh. Bộ cấp phát bắt đầu với không có bộ nhớ nào; các lời gọi `kfree` này cung cấp cho nó một ít để quản lý.

> **Ví dụ minh họa – khởi tạo free list: ví dụ số liệu**
>
> Giả sử hết phần kernel (ký hiệu `end` do linker tạo) nằm ở `0x80021234`. `freerange(end, PHYSTOP)` làm: `p = PGROUNDUP(0x80021234) = 0x80022000` (làm tròn **lên** bội số của 4096 kế tiếp; công thức `PGROUNDUP(a) = (a + 4095) & ~4095`). Sau đó với mỗi `p = 0x80022000, 0x80023000, ..., 0x87FFF000` gọi `kfree(p)`. Số trang được đưa vào danh sách: (0x88000000 − 0x80022000) / 0x1000 = **32.734 trang** (khoảng 128 MB trừ phần kernel). Vì mỗi `kfree` chèn vào **đầu** danh sách và địa chỉ được duyệt tăng dần, nên sau cùng đầu danh sách là trang **cao nhất** `0x87FFF000`; do đó các lần `kalloc` đầu tiên trả về các trang ở sát đỉnh RAM (0x87FFF000, 0x87FFE000, ...). Đây cũng là lý do các ví dụ ở mục 3.1 đặt các trang page-table ở vùng 0x87F6xxxx (sát đỉnh RAM).

Bộ cấp phát đôi khi coi địa chỉ là số nguyên để thực hiện phép tính số học trên chúng (ví dụ duyệt qua mọi trang trong `freerange`), và đôi khi dùng địa chỉ như con trỏ để đọc và ghi bộ nhớ (ví dụ thao tác với cấu trúc `run` lưu trong mỗi trang); việc dùng địa chỉ theo hai cách này là lý do chính khiến mã của bộ cấp phát đầy các phép ép kiểu (type cast) của C.

Hàm `kfree` (kernel/kalloc.c:47) bắt đầu bằng việc đặt mọi byte trong vùng bộ nhớ đang được giải phóng thành giá trị 1. Điều này sẽ khiến mã dùng bộ nhớ sau khi đã giải phóng (dùng các "dangling reference", tham chiếu treo) đọc ra rác thay vì nội dung hợp lệ cũ; hy vọng điều đó sẽ làm mã như vậy hỏng nhanh hơn. Sau đó `kfree` chèn trang vào đầu danh sách rảnh: nó ép kiểu `pa` thành con trỏ tới `struct run`, ghi đầu cũ của danh sách rảnh vào `r->next`, và đặt danh sách rảnh bằng `r`. `kalloc` gỡ và trả về phần tử đầu tiên của danh sách rảnh.

Giải thích thêm – kfree và kalloc: thuật toán và ví dụ minh họa

```
kfree(pa):                          kalloc():
    memset(pa, 1, 4096)                 r = freelist
    r = (struct run*)pa                 nếu r:  freelist = r->next
    r->next  = freelist                 nếu r:  memset(r, 5, 4096)   # xóa bằng
'rác' 5
    freelist = r                        trả về r      # NULL nếu hết bộ nhớ
```

Cả hai thao tác đều là **O(1)** (chỉ đổi vài con trỏ) – kfree là "push" và kalloc là "pop" của một **ngăn xếp** (stack, LIFO) các trang. Hình dưới minh họa: danh sách ban đầu có 3 trang; `kfree(0x87FF9000)` đẩy trang mới lên đầu; `kalloc()` kế tiếp lại lấy chính trang đó ra.

Vì sao tô rác? `kfree` tô byte `0x01` và `kalloc` tô byte `0x05`. Giả sử một lỗi lập trình: kernel `kfree(p)` rồi vẫn đọc `p->size`. Nếu trang còn nguyên dữ liệu cũ, lỗi này có thể "lặng lẽ chạy đúng" trong nhiều lần thử và chỉ phát nổ rất lâu sau đó. Với việc tô rác, giá trị đọc được là `0x0101010101010101` – một số khổng lồ vô lý – nên chương trình sai ngay lần đầu và dễ phát hiện. Tô rác cũng tránh làm lộ dữ liệu cũ sang tiến trình khác.

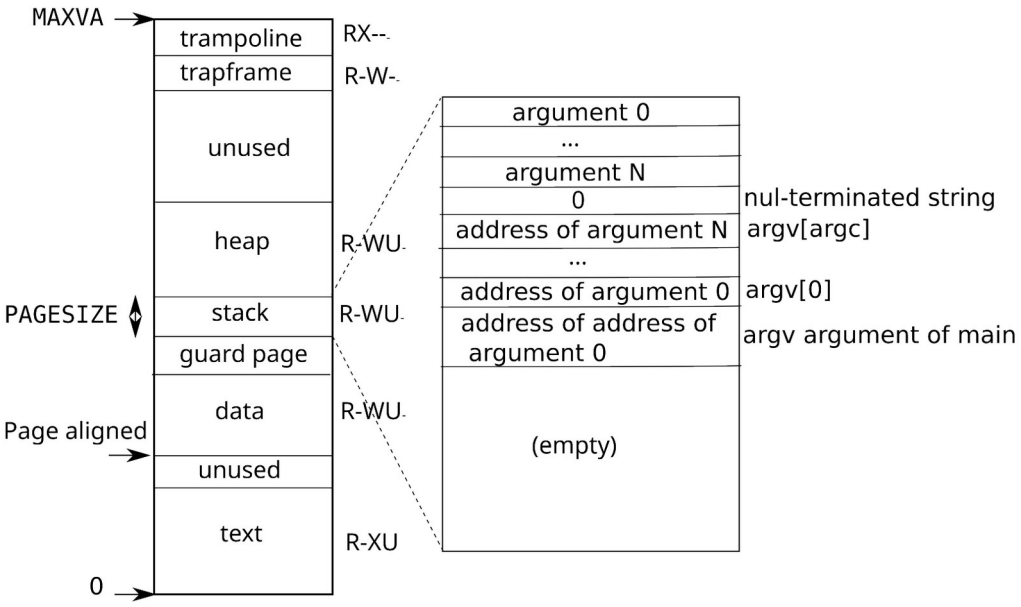


Hình bổ sung (người dịch vẽ): free list qua kfree và kalloc. Địa chỉ trang là giả định để minh họa.

3.6 Không gian địa chỉ của tiến trình (Process address space)

Mỗi tiến trình có page table riêng, và khi xv6 chuyển giữa các tiến trình, nó cũng đổi page table. Hình 3.4 cho thấy không gian địa chỉ của một tiến trình chi tiết hơn Hình 2.3. Bộ nhớ user của một tiến trình bắt đầu từ địa chỉ ảo không và có thể lớn đến `MAXVA` (kernel/riscv.h:375), về nguyên tắc cho phép một tiến trình đánh địa chỉ 256 Gigabyte bộ nhớ.

Không gian địa chỉ của một tiến trình gồm các trang chứa phần text (mã lệnh) của chương trình (xv6 ánh xạ với quyền `PTE_R`, `PTE_X` và `PTE_U`), các trang chứa dữ liệu được khởi tạo sẵn của chương trình, một trang cho stack, và các trang cho heap. Xv6 ánh xạ data, stack và heap với quyền `PTE_R`, `PTE_W` và `PTE_U`.



Hình 3.4: Không gian địa chỉ user của một tiến trình, với stack ban đầu.

Giải thích hình – đọc Hình 3.4 (từ dưới lên trên)

Cột trái là toàn bộ không gian địa chỉ ảo của một tiến trình, địa chỉ 0 ở đáy, MAXVA ở đỉnh. Cột chữ phải (R-XU, R-WU, ...) là quyền của PTE (R, W, X, U; dấu "-" nghĩa là không có quyền đó).

Vùng (dưới → trên)	Quyền	Nội dung
text	R-XU	Mã lệnh chương trình, nạp từ file ELF; không ghi được.
unused	–	Khoảng trống để căn data lên ranh giới trang ("Page aligned").
data	R-WU	Biến toàn cục đã khởi tạo + biến toàn cục (bss) – ghi được, không thực thi được.
guard page	(không có U)	Trang canh gác: PTE_V = 1 nhưng **PTE_U = 0** nên user truy cập sẽ lỗi.
stack	R-WU	Đúng **1 trang** (PAGESIZE = 4096 byte); mọc xuống về phía guard page.
heap	R-WU	Mở rộng bằng `sbrk` (mọc lên trên); ban đầu rỗng.
unused	–	Phần lớn không gian địa chỉ không dùng (PTE_V = 0).
trapframe	R-W-	Nơi lưu thanh ghi khi trap; kernel dùng; user không truy cập được (không có U).
trampoline	R-X-	Mã vào/ra kernel; dùng chung mọi tiến trình; user không chạy trực tiếp (không có U).

Cột phải (phóng to) là nội dung **trang stack ngay khi `exec` vừa xong**, đọc từ đỉnh (trên) xuống: các chuỗi đối số (argument 0 … argument N, mỗi chuỗi kết thúc bằng ký tự NUL); một số `0` (con trỏ NULL kết thúc mảng argv, tức `argv[argc]`); rồi mảng con trỏ `argv[0] … argv[argc-1]` trỏ tới các chuỗi phía trên; dòng cuối "address of address of argument 0" chính là giá trị `argv` truyền cho `main` (con trỏ trỏ tới phần tử đầu mảng). Phần còn lại phía dưới của trang là "(empty)" – vùng stack trống cho chương trình dùng.

Dùng quyền bên trong một không gian địa chỉ user là kỹ thuật phổ biến để làm cứng (harden) một tiến trình user. Nếu text được ánh xạ với `PTE_W`, thì một tiến trình có thể vô tình sửa chương trình của chính nó; ví dụ, một lỗi lập trình có thể khiến chương trình ghi vào con trỏ null, sửa đổi các lệnh tại địa chỉ 0, rồi tiếp tục chạy, có thể gây thêm hỗn loạn. Để phát hiện ngay những lỗi như vậy, xv6 ánh xạ text không có `PTE_W`; nếu một chương trình vô tình cố store vào địa chỉ 0, phần cứng sẽ từ chối thực hiện lệnh store và phát sinh page fault (xem Mục 4.6). Kernel khi đó giết tiến trình và in ra một thông báo hữu ích để lập trình viên lần ra vấn đề.

Tương tự, bằng cách ánh xạ data không có `PTE_X`, một chương trình user không thể vô tình nhảy tới một địa chỉ trong phần dữ liệu của chương trình và bắt đầu thực thi tại đó.

Ví dụ minh họa – ví dụ: lỗi con trỏ null bị bắt như thế nào

```
int *p = 0;       // con trỏ null
*p = 42;          // store vào địa chỉ ảo 0
```

Địa chỉ ảo 0 nằm trong trang **text** (R-XU, **không có W**). Phần cứng dịch địa chỉ 0 thành công, nhưng khi kiểm tra cờ thì thấy đây là lệnh **store** mà PTE_W = 0 → phát sinh **store page fault**. Kernel (Chương 4) nhận trap, thấy lỗi từ user, in một dòng chẩn đoán (kiểu `usertrap(): unexpected scause ... pid=3`) rồi hủy tiến trình. Nếu text cho phép ghi, lệnh trên sẽ lặng lẽ ghi đè 4 byte mã lệnh đầu của chương trình, và chương trình sẽ hỏng ở một chỗ rất xa lỗi gốc, rất khó gỡ lỗi.

Trong thế giới thực, làm cứng một tiến trình bằng cách đặt quyền cẩn thận cũng giúp phòng thủ trước các cuộc tấn công bảo mật. Kẻ tấn công có thể đưa vào một chương trình (ví dụ một máy chủ Web) đầu vào được dựng cẩn thận để kích hoạt một lỗi trong chương trình, với hy vọng biến lỗi đó thành một exploit [14]. Đặt quyền cẩn thận và các kỹ thuật khác, như ngẫu nhiên hóa bố cục không gian địa chỉ user, khiến các cuộc tấn công như vậy khó hơn.

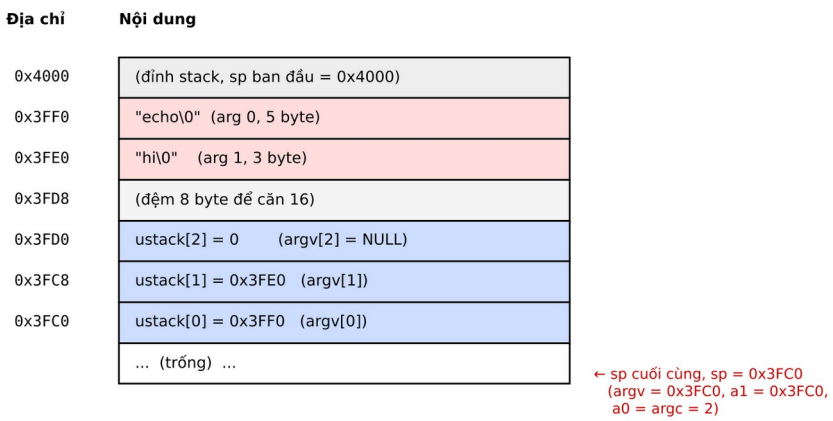
Stack là một trang duy nhất, và được hiển thị cùng nội dung ban đầu do `exec` tạo ra. Các chuỗi chứa đối số dòng lệnh, cùng một mảng con trỏ tới chúng, nằm ở ngay đỉnh của stack. Ngay bên dưới đó là các giá trị cho phép chương trình bắt đầu tại `main` như thể hàm `main(argc, argv)` vừa mới được gọi.

Ví dụ minh họa – ví dụ: stack ban đầu của lệnh `echo hi`

Giả sử chương trình có text ở 0x0000–0x0FFF và data ở 0x1000–0x1FFF; `exec` đặt guard ở 0x2000–0x2FFF và stack ở 0x3000–0x3FFF nên `sp` khởi đầu = 0x4000. `exec` đẩy lần lượt các chuỗi (mỗi chuỗi được căn 16 byte), rồi mảng con trỏ:

Bước	Việc làm	sp sau bước
1	Đẩy `"echo\0"` (5 byte): 0x4000−5 = 0x3FFB, căn xuống bội 16 → 0x3FF0; ghi `ustack[0] = 0x3FF0`	0x3FF0
2	Đẩy `"hi\0"` (3 byte): 0x3FF0−3 = 0x3FED → 0x3FE0; ghi `ustack[1] = 0x3FE0`	0x3FE0
3	`ustack[2] = 0` (NULL kết thúc argv)	0x3FE0
4	Dành chỗ cho 3 con trỏ (3 × 8 = 24 byte): 0x3FE0−24 = 0x3FC8, căn xuống bội 16 → 0x3FC0; chép `ustack[0..2]` tới 0x3FC0	0x3FC0
5	Đặt `trapframe->sp = 0x3FC0`, `trapframe->a1 = 0x3FC0` (argv); `exec` trả về argc = 2 (→ thanh ghi `a0`)	0x3FC0

Kết quả được vẽ ở hình dưới. Khi chương trình bắt đầu chạy, `main(argc, argv)` nhận `argc = 2` (từ a0) và `argv = 0x3FC0` (từ a1), nên `argv[0]` đọc ra 0x3FF0 → chuỗi “echo”, `argv[1]` → “hi”, `argv[2] == NULL`. (Giá trị địa chỉ chỉ mang tính minh họa theo thuật toán của `exec`; địa chỉ thực tế phụ thuộc kích thước text/data của từng chương trình.)



Hình bổ sung (người dịch vẽ): nội dung trang stack ban đầu cho lệnh `echo hi`.

Để phát hiện việc stack của user tràn ra ngoài vùng nhớ stack đã cấp, xv6 đặt một trang canh gác (guard page) không thể truy cập ngay bên dưới stack bằng cách xóa cờ `PTE_U`. Nếu stack của user bị tràn và tiến trình cố dùng một địa chỉ bên dưới stack, phần cứng sẽ sinh ra một ngoại lệ page-fault vì guard page không thể truy cập được đối với chương trình đang chạy ở

chế độ user. Một hệ điều hành thực tế có thể thay vào đó tự động cấp thêm bộ nhớ cho stack của user khi nó tràn.

Khi một tiến trình yêu cầu xv6 cấp thêm bộ nhớ user, xv6 mở rộng heap của tiến trình. Trước tiên xv6 dùng `kalloc` để cấp phát các trang vật lý. Sau đó nó thêm các PTE vào page table của tiến trình trỏ tới các trang vật lý mới. Xv6 đặt các cờ `PTE_W`, `PTE_R`, `PTE_U` và `PTE_V` trong các PTE này. Hầu hết các tiến trình không dùng toàn bộ không gian địa chỉ user; xv6 để `PTE_V` bị xóa trong các PTE không dùng.

Ở đây ta thấy một vài ví dụ đẹp về việc sử dụng page table. Thứ nhất, page table của các tiến trình khác nhau dịch địa chỉ user sang các trang bộ nhớ vật lý khác nhau, nhờ đó mỗi tiến trình có bộ nhớ user riêng tư. Thứ hai, mỗi tiến trình thấy bộ nhớ của mình có các địa chỉ ảo liên tục bắt đầu từ không, trong khi bộ nhớ vật lý của tiến trình có thể không liên tục. Thứ ba, kernel ánh xạ một trang chứa mã trampoline ở đỉnh không gian địa chỉ user (không có `PTE_U`), do đó một trang bộ nhớ vật lý duy nhất xuất hiện trong mọi không gian địa chỉ, nhưng chỉ kernel mới có thể dùng nó.

Ví dụ minh họa – ba ví dụ trên bằng số liệu

	Tiến trình A	Tiến trình B
Địa chỉ ảo	0x1000 (data)	0x1000 (data)
Trang vật lý (qua page table riêng)	0x87FF7000	0x87FF2000
Ý nghĩa	Cùng địa chỉ ảo, **khác** trang vật lý → bộ nhớ riêng tư; A không thể truy cập trang của B.	

Ví dụ “liên tục ảo, rời rạc vật” – một tiến trình có 3 trang ảo kề nhau 0x0000, 0x1000, 0x2000 có thể nằm ở các trang vật lý rời rạc 0x87FF9000, 0x87FDA000, 0x87FEE000: chương trình vẫn thấy một mảng liên tục, còn kernel không phải tìm 3 trang vật lý kề nhau (tránh phân mảnh ngoài). Và trampoline ở 0x3FFFFFF000 trong **mọi** page table đều trỏ tới cùng một trang vật lý duy nhất, nhưng vì PTE_U = 0 nên chỉ chạy được khi CPU ở chế độ kernel.

3.7 Mã: sbrk (Code: sbrk)

`sbrk` là system call để một tiến trình thu nhỏ hoặc mở rộng bộ nhớ của nó. System call này được cài đặt bởi hàm `growproc` (kernel/proc.c:260). `growproc` gọi `uvmalloc` hoặc `uvmdealloc`, tùy thuộc vào việc `n` là dương hay âm. `uvmalloc` (kernel/vm.c:233) cấp phát bộ nhớ vật lý bằng `kalloc`, xóa bộ nhớ đã cấp phát về 0, và thêm các PTE vào page table của user bằng `mappages`. `uvmdealloc` gọi `uvmunmap` (kernel/vm.c:178), hàm dùng `walk` để tìm các PTE và `kfree` để giải phóng bộ nhớ vật lý mà chúng tham chiếu tới.

Giải thích thêm – sbrk(n) hoạt động ra sao – thuật toán và ví dụ

```
growproc(n):                              uvmalloc(pt, oldsz, newsz):
    sz = p->sz                                for a = PGROUNDUP(oldsz); a < newsz; a +=
4096:
    nếu n > 0:  sz = uvmalloc(pt, sz, sz+n)      mem = kalloc();  nếu NULL -> thu
hồi, báo lỗi
    nếu n < 0:  sz = uvmdealloc(pt, sz, sz+n)    memset(mem, 0, 4096)        # xóa
về 0
    p->sz = sz                                    mappages(pt, a, 4096, mem, PTE_R|
PTE_W|PTE_U)
                                          trả về newsz
```

`p->sz` là kích thước (byte) của bộ nhớ user, tức biên trên của heap. **Ví dụ 1:** `sz = 0x5000`, gọi `sbrk(8192)`: `a` chạy 0x5000 và 0x6000 (2 lần < newsz = 0x7000) → cấp 2 trang vật lý, mỗi trang xóa về 0 và gắn vào PTE ảo 0x5000 và 0x6000 với cờ `R|W|U|V`; `p->sz = 0x7000`. **Ví dụ 2 (hay gặp trong Bài tập 2):** `sz = 0x5010`, gọi `sbrk(1)`: `PGROUNDUP(0x5010) = 0x6000 ≥ newsz = 0x5011` nên **vòng lặp không chạy lần nào** – byte mới nằm trong trang đã có (0x5000–0x5FFF), kernel không cấp thêm trang. Ngược lại nếu `sz = 0x6000` thì `sbrk(1)` làm `newsz = 0x6001` và kernel phải cấp **cả một trang 4096 byte** cho 1 byte. **Ví dụ 3:** `sbrk(-4096)` với `sz = 0x7000`: `uvmdealloc` gỡ ánh xạ trang 0x6000 (walk tìm PTE → `kfree` trang vật lý → xóa PTE), `p->sz = 0x6000`.

Xv6 dùng page table của một tiến trình không chỉ để cho phần cứng biết cách ánh xạ các địa chỉ ảo của user, mà còn như **bản ghi duy nhất** về những trang bộ nhớ vật lý nào đã được cấp phát cho tiến trình đó. Đó là lý do việc giải phóng bộ nhớ user (trong `uvmunmap`) đòi hỏi phải xem xét page table của user.

Giải thích thêm – hệ quả của việc "page table là sổ đăng ký duy nhất"

Xv6 không có bảng riêng kiểu "trang vật lý X thuộc về tiến trình nào". Muốn biết tiến trình P đang giữ những trang nào, kernel phải duyệt page table của P và xem mọi PTE lá hợp lệ. Vì vậy khi tiến trình kết thúc, `uvmfree` duyệt page table, `kfree` từng trang vật lý rồi mới `kfree` các trang page-table. Ưu điểm là không phải đồng bộ thêm một cấu trúc dữ liệu thứ hai; nhược điểm là truy vấn ngược (từ trang vật lý ra chủ sở hữu) rất chậm – hệ điều hành thực tế phải thêm cấu trúc như bảng "frame table"/"reverse mapping" để làm các việc như chia sẻ trang hay thu hồi trang.

3.8 Mã: exec (Code: exec)

`exec` là system call thay thế không gian địa chỉ user của một tiến trình bằng dữ liệu đọc từ một tệp, gọi là tệp nhị phân (binary) hay tệp thực thi (executable). Một binary thường là đầu ra của trình biên dịch và trình liên kết (linker), và chứa các lệnh máy và dữ liệu chương trình. `exec` (kernel/exec.c:23) mở đường dẫn của binary được đặt tên bằng `namei` (kernel/exec.c:36), được giải thích ở Chương 8. Sau đó, nó đọc phần đầu ELF (ELF header). Các binary của xv6 có định dạng ELF được dùng rộng rãi, được định nghĩa trong (kernel/elf.h). Một binary ELF gồm một ELF header, `struct elfhdr` (kernel/elf.h:6), theo

sau là một dãy các program section header, `struct proghdr` (kernel/elf.h:25). Mỗi `proghdr` mô tả một phần của ứng dụng cần được nạp vào bộ nhớ; các chương trình xv6 có hai program section header: một cho các lệnh và một cho dữ liệu.

Bước đầu tiên là một kiểm tra nhanh xem tệp có khả năng chứa một binary ELF hay không. Một binary ELF bắt đầu bằng "số ma thuật" (magic number) bốn byte 0x7F, 'E', 'L', 'F', hay `ELF_MAGIC` (kernel/elf.h:3). Nếu ELF header có số ma thuật đúng, `exec` giả định binary được cấu trúc đúng.

> **Giải thích thêm – bố cục tệp ELF và số ma thuật**
>
> Một tệp ELF giống như một "bản hướng dẫn lắp ráp": phần đầu (ELF header) ghi **entry** (địa chỉ lệnh đầu tiên cần chạy), số lượng và vị trí các program header; mỗi program header nói: "lấy `filesz` byte tại vị trí `off` trong tệp, đặt vào bộ nhớ ảo tại `vaddr`, và dành tổng cộng `memsz` byte với quyền `flags`".
>
> Số ma thuật: 4 byte đầu tệp là `7F 45 4C 46` (0x45 = 'E', 0x4C = 'L', 0x46 = 'F'). Vì RISC-V là little-endian, khi đọc 4 byte này thành một số `uint` 32 bit ta được `0x464C457F` – đây chính là giá trị `ELF_MAGIC` trong mã (`0x7FELF` viết kiểu "little-endian"). Nếu 4 byte đầu khác (ví dụ một tệp văn bản), `exec` đi tới nhãn `bad` và trả về −1.

`exec` cấp phát một page table mới không có ánh xạ user nào bằng `proc_pagetable` (kernel/exec.c:49), cấp phát bộ nhớ cho mỗi segment ELF bằng `uvmalloc` (kernel/exec.c:65), và nạp từng segment vào bộ nhớ bằng `loadseg` (kernel/exec.c:10). `loadseg` dùng `walkaddr` để tìm địa chỉ vật lý của bộ nhớ đã cấp phát, nơi ghi từng trang của segment ELF, và dùng `readi` để đọc từ tệp.

Program section header của `/init`, chương trình user đầu tiên được tạo bằng `exec`, trông như sau:

```
# objdump -p user/_init

user/_init:         file format elf64-little

Program Header:
0x70000003 off    0x0000000000006bb0 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**0
         filesz 0x000000000000004a memsz 0x0000000000000000 flags r--
    LOAD off    0x0000000000001000 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**12
         filesz 0x0000000000001000 memsz 0x0000000000001000 flags r-x
    LOAD off    0x0000000000002000 vaddr 0x0000000000001000
                                     paddr 0x0000000000001000 align 2**12
         filesz 0x0000000000000010 memsz 0x0000000000000030 flags rw-
   STACK off    0x0000000000000000 vaddr 0x0000000000000000
                                     paddr 0x0000000000000000 align 2**4
         filesz 0x0000000000000000 memsz 0x0000000000000000 flags rw-
```

Ta thấy rằng segment text cần được nạp tại địa chỉ ảo 0x0 trong bộ nhớ (không có quyền ghi) từ nội dung tại offset 0x1000 trong tệp. Ta cũng thấy rằng data cần được nạp tại địa chỉ 0x1000, nằm đúng ranh giới trang, và không có quyền thực thi.

Ví dụ minh họa – đọc từng dòng của bảng objdump trên

Dòng	off (trong tệp)	vaddr (ảo)	filesz / memsz	flags	Xử lý của exec
0x70000003	0x6bb0	0x0	0x4a / 0	r--	Loại đặc biệt của RISC-V (thuộc tính), **không phải LOAD** → exec bỏ qua.
LOAD #1 (text)	0x1000	0x0	0x1000 / 0x1000	r-x	Cấp 1 trang tại 0x0, đọc 0x1000 byte từ tệp; map với R|X|U.
LOAD #2 (data)	0x2000	0x1000	0x10 / 0x30	rw-	Cấp 1 trang tại 0x1000; đọc 0x10 byte từ tệp; 0x20 byte còn lại = 0; map R|W|U.
STACK	0x0	0x0	0 / 0	rw-	Chỉ là gợi ý về stack, bỏ qua (exec tự tạo stack).

Chỉ các program header kiểu `LOAD` mới được nạp (`if (ph.type != ELF_PROG_LOAD) continue;`). Cột `align 2**12` nghĩa là `vaddr` phải là bội số của 4096 – nên xv6 kiểm tra `ph.vaddr % PGSIZE == 0`. Lưu ý: tuy đã có cờ r-x/rw- trong tệp, xv6 tự suy ra quyền PTE từ cờ ELF (`flags & ELF_PROG_FLAG_EXEC` → thêm PTE_X, `flags & ELF_PROG_FLAG_WRITE` → thêm PTE_W).

`filesz` của một program section header có thể nhỏ hơn `memsz`, cho biết khoảng trống giữa chúng cần được lấp bằng số không (cho các biến toàn cục của C) thay vì được đọc từ tệp. Với `/init`, `filesz` của data là 0x10 byte còn `memsz` là 0x30 byte, và do đó `uvmalloc` cấp phát đủ bộ nhớ vật lý để chứa 0x30 byte, nhưng chỉ đọc 0x10 byte từ tệp `/init`.

Giải thích thêm – vì sao filesz < memsz? (.data và .bss)

Ví dụ trong C: `int a = 5;` (biến toàn cục có giá trị khởi tạo) nằm trong **.data** – giá trị 5 phải được lưu trong tệp. `int buf[8];` (toàn cục, không khởi tạo) nằm trong **.bss** – theo chuẩn C nó phải bằng 0, nên tệp **không cần lưu** 32 byte số 0 mà chỉ ghi "cần thêm 32 byte". Với `/init`: 0x10 byte đầu là `.data` thật (đọc từ tệp), 0x20 byte sau là `.bss` (0x30 − 0x10 = 0x20). `uvmalloc` đã **xóa trang về 0** nên không cần làm gì thêm cho phần bss. Nhờ đó tệp thực thi nhỏ hơn.

Bây giờ `exec` cấp phát và khởi tạo stack của user. Nó chỉ cấp phát một trang stack. `exec` sao chép lần lượt từng chuỗi đối số vào đỉnh stack, ghi lại các con trỏ tới chúng trong `ustack`. Nó đặt một con trỏ null ở cuối thứ sẽ trở thành danh sách `argv` được truyền cho `main`. Các giá trị cho `argc` và `argv` được truyền cho `main` qua đường trở về của system call: `argc` được truyền qua giá trị trả về của system call, được đặt vào `a0`, và `argv` được truyền qua phần tử `a1` của trapframe của tiến trình.

Giải thích thêm – vì sao argc ở a0 còn argv ở a1?

`exec` là một system call nên sau khi chạy xong nó "quay về" user bằng đường trở về thông thường của system call: giá trị trả về của hàm `exec` (chính là `argc`) được kernel đặt vào thanh ghi `a0` của trapframe. Mà theo quy ước gọi hàm của RISC-V, `a0` và `a1` là hai đối số đầu của hàm: khi chương trình mới bắt đầu ở `main(int argc, char *argv[])`, `a0` đã có sẵn `argc`. Vì không có giá trị trả về thứ hai, `exec` tự tay ghi `p->trapframe->a1 = sp` để `argv` có mặt trong `a1`. Nhờ trick này mà `main` nhận đúng hai đối số mà không cần mã "khởi động" (startup code) đặc biệt.

`exec` đặt một trang không thể truy cập ngay bên dưới trang stack, để những chương trình cố dùng nhiều hơn một trang sẽ gặp lỗi (fault). Trang không thể truy cập này cũng cho phép `exec` xử lý các đối số quá lớn; trong tình huống đó, hàm `copyout` (kernel/vm.c:359) mà `exec` dùng để sao chép đối số vào stack sẽ nhận thấy trang đích không thể truy cập được, và sẽ trả về −1.

Trong quá trình chuẩn bị ảnh bộ nhớ (memory image) mới, nếu `exec` phát hiện một lỗi như một program segment không hợp lệ, nó nhảy tới nhãn `bad`, giải phóng ảnh mới, và trả về −1. `exec` phải chờ để giải phóng ảnh cũ cho đến khi chắc chắn system call sẽ thành công: nếu ảnh cũ đã mất, system call không thể trả về −1 cho nó. Các trường hợp lỗi duy nhất trong `exec` xảy ra trong quá trình tạo ảnh. Khi ảnh đã hoàn chỉnh, `exec` có thể cam kết (commit) với page table mới (kernel/exec.c:125) và giải phóng cái cũ (kernel/exec.c:129).

Giải thích thêm – tóm tắt luồng exec và nguyên tắc "dựng cái mới trước, bỏ cái cũ sau"

1. `namei(path)` mở tệp → `readi` đọc ELF header → kiểm tra số ma thuật (sai → `bad`).
2. `proc_pagetable(p)` tạo page table **mới** (chỉ có trampoline + trapframe, chưa có ánh xạ user). Page table **cũ vẫn còn nguyên** trong `p->pagetable`.
3. Với mỗi program header kiểu LOAD: kiểm tra hợp lệ → `uvmalloc` cấp trang → `loadseg` đọc dữ liệu từ tệp vào trang (qua `walkaddr` + `readi`).
4. Cấp 2 trang cho stack (1 guard + 1 stack), dùng `uvmclear` bỏ PTE_U của trang guard; chép đối số vào stack như ví dụ `echo hi` ở mục 3.6.
5. **Commit:** `p->pagetable = pagetable` (page table mới), `p->sz = sz`, `trapframe->epc = elf.entry` (điểm bắt đầu chương trình), `trapframe->sp = sp`; rồi `proc_freepagetable(oldpagetable)` giải phóng ảnh cũ. Trả về `argc`.

Vì sao commit sau cùng? Giả sử ta giải phóng ảnh cũ ngay ở bước 2 rồi ở bước 3 phát hiện tệp ELF hỏng. Lúc này tiến trình không còn code/data nào để chạy tiếp, mà `exec` lại phải "trả về −1 cho chương trình gọi nó" – nhưng chương trình gọi đã bị xóa! Với cách làm của xv6, nếu lỗi xảy ra trước bước 5, ta chỉ giải phóng thứ đang dựng dở (page table mới), tiến trình cũ vẫn nguyên vẹn và nhận −1 như bình thường (ví dụ shell in "exec failed" và tiếp tục chạy). Đây là nguyên tắc *tất cả hoặc không gì cả* (atomicity) ở mức thao tác hệ thống.

`exec` nạp các byte từ tệp ELF vào bộ nhớ tại các địa chỉ do tệp ELF chỉ định. Người dùng hoặc các tiến trình có thể đặt bất kỳ địa chỉ nào họ muốn vào một tệp ELF. Do đó `exec` có rủi

ro, vì các địa chỉ trong tệp ELF có thể tham chiếu tới kernel, vô tình hoặc cố ý. Hậu quả đối với một kernel bất cẩn có thể từ một vụ crash đến việc phá hoại độc hại các cơ chế cô lập của kernel (tức là một exploit bảo mật). Xv6 thực hiện một số kiểm tra để tránh các rủi ro này. Ví dụ, `if(ph.vaddr + ph.memsz < ph.vaddr)` kiểm tra xem phép cộng có tràn số nguyên 64 bit hay không. Nguy cơ là người dùng có thể dựng một binary ELF với `ph.vaddr` trỏ tới một địa chỉ do người dùng chọn, và `ph.memsz` đủ lớn để tổng tràn về 0x1000, giá trị này sẽ trông như hợp lệ. Trong một phiên bản xv6 cũ hơn, nơi không gian địa chỉ user cũng chứa kernel (nhưng không đọc/ghi được ở chế độ user), người dùng có thể chọn một địa chỉ tương ứng với bộ nhớ kernel và do đó sao chép dữ liệu từ binary ELF vào kernel. Trong phiên bản RISC-V của xv6 điều này không thể xảy ra, vì kernel có page table riêng biệt; `loadseg` nạp vào page table của tiến trình, không phải page table của kernel.

> **Ví dụ minh họa – ví dụ tấn công tràn số (integer overflow) và cách kiểm tra chặn nó**
>
> Kẻ tấn công tạo tệp ELF với `ph.vaddr = 0xFFFFFFFFFFFFF000` và `ph.memsz = 0x2000`. Phép cộng 64 bit: 0xFFFFFFFFFFFFF000 + 0x2000 = 0x1_0000_0000_0000_1000 → bị **cắt về 64 bit** thành `0x1000` (tràn số – "wrap around"). Nếu mã kiểm tra chỉ xem `vaddr + memsz` có nằm trong giới hạn cho phép (ví dụ nhỏ hơn MAXVA) hay không, kết quả `0x1000` trông rất hợp lệ và qua mặt được kiểm tra, trong khi `vaddr` thực sự nằm ở vùng cao bất hợp lệ. Điều kiện `ph.vaddr + ph.memsz < ph.vaddr` bắt đúng trường hợp này: kết quả sau khi tràn (0x1000) **nhỏ hơn** `vaddr` (0xFFFFFFFFFFFFF000), điều không bao giờ xảy ra với phép cộng không tràn của hai số không âm → xv6 từ chối tệp (nhảy tới `bad`).
>
> Danh sách các kiểm tra trong vòng lặp nạp segment của xv6: kiểu phải là LOAD; `memsz >= filesz`; `vaddr + memsz` không tràn; `vaddr` phải căn theo trang. Mỗi cái chặn một kiểu đầu vào ác ý khác nhau.

Một lập trình viên kernel rất dễ bỏ sót một kiểm tra quan trọng, và các kernel trong thế giới thực có một lịch sử dài những kiểm tra bị thiếu mà việc vắng mặt của chúng có thể bị các chương trình user khai thác để chiếm đặc quyền kernel. Nhiều khả năng xv6 không làm đầy đủ việc kiểm tra tính hợp lệ của dữ liệu cấp user được cung cấp cho kernel, mà một chương trình user độc hại có thể khai thác để phá vỡ sự cô lập của xv6.

3.9 Thế giới thực (Real world)

Giống như hầu hết các hệ điều hành, xv6 dùng phần cứng phân trang để bảo vệ và ánh xạ bộ nhớ. Hầu hết các hệ điều hành dùng phân trang tinh vi hơn xv6 rất nhiều bằng cách kết hợp phân trang với các ngoại lệ page-fault, mà ta sẽ thảo luận ở Chương 4.

Giải thích thêm – phân trang + page fault cho phép những gì (xem trước Chương 4)

- **Cấp phát lười (lazy allocation):** `sbrk(1 GB)` chỉ tăng `p->sz`, chưa cấp trang nào; trang được cấp thật khi chương trình lần đầu chạm tới (gây page fault).
- **Copy-on-write fork:** cha và con ban đầu cùng ánh xạ các trang vật lý như nhau (chỉ đọc); chỉ khi một bên ghi, kernel mới sao chép trang đó.
- **Demand paging / swap:** nạp trang từ tệp/đĩa khi cần, hoặc đẩy trang ít dùng ra đĩa khi RAM cạn.
- **Stack tự mở rộng, mmap tệp** và nhiều tính năng khác đều dựa trên cùng cơ chế: PTE không hợp lệ → page fault → kernel quyết định xử lý.

Xv6 được đơn giản hóa nhờ việc kernel dùng ánh xạ trực tiếp giữa địa chỉ ảo và địa chỉ vật lý, và nhờ giả định rằng có RAM vật lý tại địa chỉ 0x80000000, nơi kernel mong đợi được nạp. Điều này hoạt động với QEMU, nhưng trên phần cứng thật thì hóa ra là một ý tưởng tồi; phần cứng thật đặt RAM và các thiết bị tại các địa chỉ vật lý không thể đoán trước, nên (ví dụ) có thể không có RAM tại 0x80000000, nơi xv6 mong đợi có thể lưu kernel. Các thiết kế kernel nghiêm túc hơn khai thác page table để biến các bố cục bộ nhớ vật lý phần cứng tùy ý thành các bố cục địa chỉ ảo kernel có thể đoán trước.

RISC-V hỗ trợ bảo vệ ở mức địa chỉ vật lý, nhưng xv6 không dùng tính năng đó.

Trên các máy có nhiều bộ nhớ, có thể hợp lý khi dùng hỗ trợ "trang siêu lớn" (super pages) của RISC-V. Các trang nhỏ hợp lý khi bộ nhớ vật lý nhỏ, để cho phép cấp phát và đẩy trang ra đĩa (page-out) với độ mịn cao. Ví dụ, nếu một chương trình chỉ dùng 8 kilobyte bộ nhớ, việc cấp cho nó cả một super-page 4 megabyte bộ nhớ vật lý là lãng phí. Các trang lớn hơn hợp lý trên các máy có nhiều RAM, và có thể giảm chi phí thao tác page table.

Giải thích thêm – super page trong Sv39 và lưu ý về con số "4 megabyte"

Trong Sv39, một PTE ở cấp **L1** có thể là PTE lá (có R/W/X) và khi đó nó ánh xạ trực tiếp một khối **2 MB** (không còn cấp L0); một PTE lá ở cấp **L2** ánh xạ khối **1 GB**. Các khối này gọi là *megapage* và *gigapage*. (Con số "4 MB" trong bản gốc tương ứng với chuẩn Sv32 của RISC-V 32 bit; với Sv39 các kích thước siêu trang là 2 MB và 1 GB.) **Ví dụ:** ánh xạ 1 GB RAM bằng trang 4 KB cần 262.144 PTE (khoảng 512 trang page-table) và TLB phải giữ rất nhiều mục; bằng một gigapage chỉ cần **1 PTE**, và một mục TLB phủ cả 1 GB. Đổi lại: nếu một chương trình chỉ cần 8 KB mà cấp cả khối 2 MB thì lãng phí gần 2 MB (phân mảnh trong).

Việc kernel xv6 thiếu một bộ cấp phát kiểu `malloc` có thể cung cấp bộ nhớ cho các đối tượng nhỏ khiến kernel không thể dùng các cấu trúc dữ liệu tinh vi đòi hỏi cấp phát động. Một kernel công phu hơn nhiều khả năng sẽ cấp phát nhiều kích cỡ khối nhỏ khác nhau, thay vì (như trong xv6) chỉ các khối 4096 byte; một bộ cấp phát kernel thực thụ cần xử lý được các cấp phát nhỏ cũng như lớn.

Ví dụ minh họa – vấn đề cấp phát khối nhỏ

Giả sử kernel cần một đối tượng 64 byte (ví dụ một node của danh sách). Với `kalloc` của xv6, để có 64 byte kernel buộc phải lấy **cả trang 4096 byte** – lãng phí 4032 byte (98%). Các hệ điều hành thực tế (Linux với *slab allocator*, BSD với *zone allocator*) lấy một trang từ bộ cấp phát trang rồi **chia nhỏ** thành nhiều khối cùng kích cỡ (64 byte → 64 khối/trang) và dùng free list riêng cho từng kích cỡ.

Cấp phát bộ nhớ là một chủ đề nóng lâu năm, các vấn đề cơ bản là sử dụng hiệu quả bộ nhớ hạn chế và chuẩn bị cho các yêu cầu chưa biết trong tương lai [9]. Ngày nay người ta quan tâm đến tốc độ hơn là hiệu quả không gian.

3.10 Bài tập (Exercises)

1. Phân tích cây thiết bị (device tree) của RISC-V để tìm lượng bộ nhớ vật lý mà máy tính có.
2. Viết một chương trình user làm tăng không gian địa chỉ của nó thêm một byte bằng cách gọi `sbrk(1)`. Chạy chương trình và khảo sát page table của chương trình trước lời gọi `sbrk` và sau lời gọi `sbrk`. Kernel đã cấp phát bao nhiêu không gian? PTE cho bộ nhớ mới chứa gì?
3. Sửa xv6 để dùng các super page cho kernel.
4. Các cài đặt Unix của `exec` theo truyền thống có xử lý đặc biệt cho các shell script. Nếu tệp cần thực thi bắt đầu bằng văn bản `#!`, thì dòng đầu tiên được coi là một chương trình để chạy nhằm thông dịch tệp. Ví dụ, nếu `exec` được gọi để chạy `myprog arg1` và dòng đầu của `myprog` là `#!/interp`, thì `exec` chạy `/interp` với dòng lệnh `/interp myprog arg1`. Hãy cài đặt hỗ trợ quy ước này trong xv6.
5. Cài đặt ngẫu nhiên hóa bố cục không gian địa chỉ (address space layout randomization – ASLR) cho kernel.

Giải thích thêm – gợi ý hướng làm cho các bài tập

- **Bài 1:** Trên QEMU, địa chỉ của device tree được bootloader truyền cho kernel qua thanh ghi `a1` khi khởi động (xem tài liệu QEMU/`-machine virt,dumpdtb=...`). Cây này ở dạng FDT; tìm node `/memory` và đọc thuộc tính `reg` (địa chỉ bắt đầu, kích thước). Nên dùng lệnh `dtc -I dtb -O dts` trên máy để xem cấu trúc trước khi viết bộ phân tích trong kernel.
- **Bài 2:** Với lời giải thích ở mục 3.7: tùy giá trị `sz` ban đầu mà `sbrk(1)` có thể không cấp thêm trang (nếu `sz` chưa tới ranh giới trang) hoặc cấp một trang 4096 byte (nếu `sz` đúng ở ranh giới). PTE mới sẽ có cờ `V|R|W|U` = 0x17 và PPN trỏ tới trang vừa `kalloc`. Có thể thêm một system call hoặc hàm debug (như `vmprint` ở lab) để in cây page table trước/sau.
- **Bài 3:** Cần map vùng kernel bằng PTE lá ở cấp L1 (2 MB, địa chỉ phải căn 2 MB) – sửa `kvmmap`/`walk` để dừng sớm một cấp; chú ý `walk` hiện luôn đi xuống cấp L0.
- **Bài 4:** Trong `exec`, sau khi đọc ELF header (hoặc đọc 2 byte đầu của tệp) kiểm tra có phải `#!` không; nếu có, đọc dòng đầu để lấy đường dẫn trình thông dịch rồi dựng lại `argv` mới = {interp, đường dẫn tệp gốc, đối số cũ...} và gọi `exec` đệ quy (hoặc lặp) với interp. Đừng quên giới hạn độ sâu để tránh vòng lặp vô hạn.
- **Bài 5:** Ý tưởng: khi dựng page table kernel, thay vì cố định `KERNBASE`, chọn một offset ngẫu nhiên (căn theo trang) để dịch ánh xạ ảo của kernel; cần mã kernel không phụ thuộc địa chỉ tuyệt đối (position-independent) hoặc phải sửa lại các con trỏ sau khi chuyển.

— Hết Chương 3 —